ASMPT

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

**Department of Electrical Engineering**

# Model Augmentation for a High-Precision Dual-Drive Gantry

by

**D.J.L.M. van den Berg**

MSc Thesis

**Thesis committee**

dr. ir. M. Schoukens
prof. dr. ir. R. Tóth
dr. ir. M.F. Shakib
ir. J. W. Gerritsen

**Graduation**

| | |
|---|---|
| Program: | Artificial Intelligence and Engineering Systems |
| Research group: | Control Systems |
| Thesis supervisor: | dr. ir. M. Schoukens |
| Date of defense: | 17-11-2026 |
| Student ID: | 1555561 |
| Study load (ECTS): | 45 |
| Track: | High-Tech Systems and Robotics |

The research of this thesis has been carried out in collaboration with ASMPT, Beuningen.

This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

# Model Augmentation for a High-Precision Dual-Drive Gantry

Dirk van den Berg
*Control Systems Group, Dept. of Electrical Engineering*
*Eindhoven University of Technology*
Eindhoven, The Netherlands
In collaboration with *ASMPT*, Beuningen, The Netherlands

Supervisors: dr. ir. M. Schoukens, prof. dr. ir. R. Tóth, ir. J.H. Hoekstra (TU/e);
ir. Q. van den Elsen, ir. J. W. Gerritsen (ASMPT)

***Abstract*—A plant model can be used to evaluate a changed feedback controller before it is tested on the machine. For the dual-drive gantry, both predictive accuracy and physical interpretability are required. However, the physics-based baseline does not capture all dynamics. When the parameters of the baseline model are refitted, they are able to absorb parts of the missing dynamics. As a result, the parameter estimates can deviate from their true values. Adding a neural network can capture the missing dynamics instead, but it can also represent dynamics that the baseline parameters can already explain. We augment the baseline in parallel with a neural network and additional states to model the missing dynamics. The added states are initialised with predictive-state (PS) initialisation, and the model is trained in closed loop with the known controller. To prevent the neural network from correcting what the baseline parameters can represent, we extend orthogonal-by-construction (OBC) augmentation to a state-space baseline nonlinear in its parameters. We evaluate the method in simulation on held-out positions and unseen motion profiles, against the baseline and a black box model. The augmented model reduces the prediction error when the missing vibration mode is sufficiently excited. OBC reduces the overlap between the neural network and the parameter directions. However, the parameter estimates still depend on their initial values.**

## I. Introduction

Semiconductor devices are essential for applications such as artificial intelligence, 5G and autonomous vehicles. Shortages can affect entire industries and national security [1]. Manufacturing these devices requires high-precision motion systems, with strict requirements on alignment accuracy, yield, throughput and cost [2], [3]. Accurate plant models help to meet these requirements [4], [5]. This thesis focuses on one such motion system: a dual-drive gantry in which two parallel linear actuators support a common beam carrying the payload. The shared beam couples the dynamics of both actuators, so the motion of one actuator influences the other and therefore affects its tracking accuracy [6], [7]. The system exhibits position-dependent behaviour and flexible dynamics [6].

A plant model can be used to evaluate a changed feedback controller before it is tested on the machine. A closed-loop simulation can be faster than an experiment, allowing multiple controllers to be compared simultaneously, and without risk of harming the hardware. For the plant model of the dual-drive gantry, both predictive accuracy and physical interpretability are required. In this thesis, physical interpretability means that the identifiable combinations of the physical parameters are estimated uniquely [8]. This does not guarantee identifying the true parameters, but a deviation can then indicate which physical parameter might have changed, making such models valuable for root cause analysis [9].

White-box models are attractive because they reflect the physics of the system, which makes them interpretable. First-principles (FP) models are a specific type of white-box model, derived from physical laws and described by quantities such as mass, stiffness, and damping. However, purely physics-based models do not always capture all behaviour needed for accurate prediction across the operating range [10]. More accurate FP models can be developed, but this is labour-intensive [5]. Purely data-driven (black-box) models such as neural-network state-space models can describe measured behaviour well, but can offer limited physical insight and are not guaranteed to be accurate at operating conditions not covered by the training data [5], [10]. In addition, these models need to learn behaviour that is already known from first principles, such as the rigid-body dynamics [5].

Physics-informed approaches, such as physics-informed or physics-guided neural networks and Gaussian-process-based methods, incorporate physical knowledge during training [11], [12], [10], but the physics-informed and physics-guided neural networks discussed by Hoekstra et al. still rely on a black-box model for the entire system [5].

Grey-box modelling is the logical choice, because it is able to combine a baseline model with interpretable parameters and a data-driven model for accurate prediction [13]. Grey-box approaches include parameter re-estimation within the physics model structure, static residual correction on the model output, and dynamic augmentation that extends the baseline with learned states [5], [10]. Model augmentation combines a physics-based baseline model with a learned component, so that the learned component only needs to capture the dynamics the physics-based model cannot describe [5], [14]. Dynamic augmentation is chosen to model missing dynamics that require additional states, which parameter re-estimation and static output correction cannot provide [5]. However, related

work on an industrial wire bonder shows that preserving physical interpretability during model updating remains an open challenge [4]. Refitting the physical parameters allows them to absorb unmodelled dynamics, and an unconstrained learned component is able to absorb dynamics the baseline already describes [8].

Dynamic augmentation has already been demonstrated, but Hoekstra et al.'s examples do not consider position-dependent baselines or training with a simulated feedback controller [5]. The related work on the wire bonder uses closed-loop data, but keeps the physical parameters fixed during training [4]. Orthogonal-by-construction (OBC) augmentation keeps the learned component orthogonal to the baseline regressor space, so that the physical parameter estimates are unique under the stated conditions [8]. Györök et al. formulate this for discrete-time input-output models, leaving extension to state-space models for future work. A key gap remains: the reviewed approaches do not simultaneously address coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour and closed-loop data while preserving physical interpretability. This thesis therefore develops such a framework for the dual-drive gantry and evaluates it in simulation.

This thesis addresses the following research question: *For the dual-drive gantry, how should a physics-based plant model be augmented to capture unmodelled dynamics, improving prediction accuracy across operating points and motion profiles while preserving the physical interpretability of its parameters?* To answer this research question, the following three aspects are investigated:

1) Which augmentation structure and training method let the learned component capture the missing dynamics from closed-loop data?
2) To what extent does orthogonal-by-construction augmentation keep the physical parameters interpretable, compared to unconstrained estimation?
3) How does the augmented model generalise to held-out operating points and unseen motion profiles, compared to the physics baseline and a black-box model?

The thesis is organised as follows: Section II states the identification problem, Section III the physics baseline, Section IV the augmented model and its closed-loop identification, Section V the orthogonal construction, Section VI the experiment design, Section VII the results and discussion, and Section VIII the conclusions.

## II. Identification Problem

We consider a dual-drive gantry with two parallel linear actuators that are connected with a cross-arm and a third linear actuator that moves a payload along this arm (Fig. 1), as in García-Herreros et al. [6]. The actuator forces are defined as $u_{\text{act}} = [F_{X1}\ F_{X2}\ F_Y]^\top$ and the measured positions $q_{\text{act}} = [X_1\ X_2\ Y]^\top$. Here $F_{X1}$, $F_{X2}$ and $F_Y$ are the forces and $X_1$, $X_2$ and $Y$ the measured positions of the two X and the Y actuators, each from a linear encoder [6].

The system to be identified is the gantry system described by

$$\dot{x}(t) = f\big(x(t), u_{\text{act}}(t)\big), \qquad y_k = q_{\text{act}}(kT_s) + v_k, \quad (1)$$

where the state $x$ contains the actuator positions $q_{\text{act}}$, $y_k$ are the measured positions at sample $k$ with sampling time $T_s$, and $v_k$ is zero-mean, coloured measurement noise. The function $f$ and the dimension of $x$ are unknown. The system runs in closed loop with an assumed known linear time-invariant feedback controller, which gives the data set $\mathcal{D}_N = \{u_{\text{act},k}, y_k\}_{k=1}^N$. The controller computes the actuator forces from the error between the reference $r_k$ and the measured positions $y_k$, and a feedforward $u_k^{\text{ff}}$ is added (Fig. 2):

$$\begin{aligned} x_{k+1}^{\text{fb}} &= A_{\text{fb}} x_k^{\text{fb}} + B_{\text{fb}}(r_k - y_k), \\ u_{\text{act},k} &= C_{\text{fb}} x_k^{\text{fb}} + D_{\text{fb}}(r_k - y_k) + u_k^{\text{ff}}, \end{aligned} \quad (2)$$

where $(A_{\text{fb}}, B_{\text{fb}}, C_{\text{fb}}, D_{\text{fb}})$ are the state-space matrices of the controller $K_{\text{fb}}$, with state $x^{\text{fb}}$, discretised with sampling time $T_s$, and the actuator forces are held constant between samples, $u_{\text{act}}(t) = u_{\text{act},k}$ for $kT_s \le t < (k+1)T_s$. Both $r_k$ and $u_k^{\text{ff}}$ are in actuator coordinates. The noise $v$ is independent of $r$ and $u^{\text{ff}}$. The force $u_{\text{act}}$ is correlated with $v$, because the controller computes it from $y$.

We use the lumped-parameter model of Fig. 1 as our physics-based baseline model of (1). This model does not capture all dynamics of the system. We identify these missing dynamics from $\mathcal{D}_N$ and augment the baseline with learned components and additional states [5].

The goal is to estimate the physical and learned parameters of the augmented model from $\mathcal{D}_N$, such that it captures the missing dynamics and accurately predicts the system, while the identifiable combinations of the physical parameters are estimated uniquely. We use the SUBNET method of Beintema et al. [15] to train the model on short windows of $\mathcal{D}_N$, where each window is simulated with the controller $K_{\text{fb}}$. The initial states are estimated by an encoder network from past inputs and outputs.

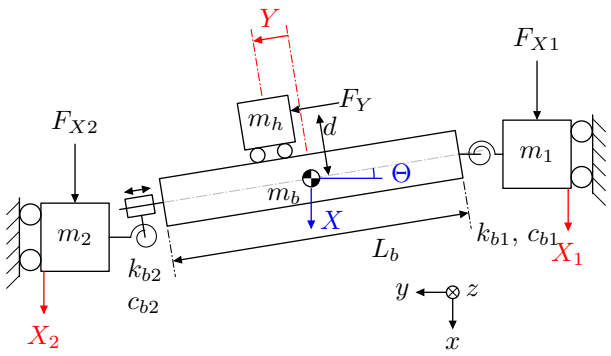



Fig. 1: Lumped-parameter model of the Dual-drive gantry stage, adapted from [6]. At $X_2$, a prismatic joint lets the cross-arm slide, allowing it to rotate. Red labels are actuator coordinates and blue labels generalised coordinates, with $Y$ in both.

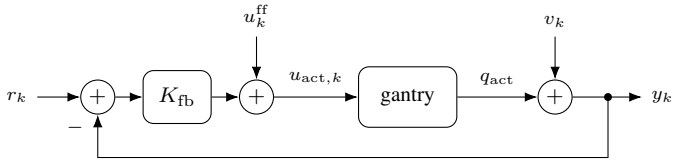



Fig. 2: Closed loop that generates the data: the controller $K_{\mathrm{fb}}$ of (2), the feedforward $u_k^{\mathrm{ff}}$, the gantry (1) and the measurement noise $v_k$.

## III. Dual-Drive Gantry Physics Baseline

In this section, we state the equations of motion of the baseline, its position-dependent state-space model, and the identifiable combinations of the physical parameters and how we parametrise them for estimation.

### A. Equations of motion

García-Herreros et al. derive the equations of motion of the model in Fig. 1 from the Euler-Lagrange equations [6, Eqs. (3) to (8)]. Let $L_b$ denote the cross-arm length between the two flexible joints. The generalised coordinates $q = [X\ \Theta\ Y]^\top$ are defined as $X = (X_1 + X_2)/2$ and $\Theta = (X_1 - X_2)/L_b$, the position and yaw angle of the cross-arm, while $Y$ is unchanged. These coordinates avoid closed kinematic chains when defining the kinematics [6]. They also show the coupling caused by the non-uniform load distribution between actuators $X_1$ and $X_2$, from the payload and differences in motor characteristics. The coordinate and force transformations are

$$q = P^{-\top} q_{\mathrm{act}}, \qquad u = P u_{\mathrm{act}}, \qquad P = \begin{bmatrix} 1 & 1 & 0 \\ L_b/2 & -L_b/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{3}$$

where $u$ denotes the generalised force. Following García-Herreros et al. [6], the baseline assumes $\cos(\Theta) \approx 1$ and $\sin(\Theta) \approx 0$, valid for small yaw angles, and assumes that the Coriolis and centripetal force terms are negligible compared to the inertia, damping, and stiffness forces. Our baseline model does not include the Coulomb friction of [6, Eq. (6)]. Instead, it is part of the missing dynamics of Section II. Under these assumptions, the baseline equations of motion are

$$M\big(Y(t)\big)\ddot{q}(t) + C\dot{q}(t) + Kq(t) = u(t), \tag{4}$$

with

$$M(Y) = \begin{bmatrix} m_1 + m_2 + m_b + m_h & \frac{L_b}{2}(m_1 - m_2) - m_h Y & 0 \\ \frac{L_b}{2}(m_1 - m_2) - m_h Y & J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2) + m_h(d^2 + Y^2) & -m_h d \\ 0 & -m_h d & m_h \end{bmatrix}, \tag{5a}$$

$$C = \begin{bmatrix} c_{g1} + c_{g2} & \frac{L_b}{2}(c_{g1} - c_{g2}) & 0 \\ \frac{L_b}{2}(c_{g1} - c_{g2}) & c_{b1} + c_{b2} + \frac{L_b^2}{4}(c_{g1} + c_{g2}) & 0 \\ 0 & 0 & c_y \end{bmatrix}, \tag{5b}$$

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & k_{b1} + k_{b2} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{5c}$$

From here on the notation for the time dependency is dropped. The masses, stiffness and damping coefficients, and dimensions $L_b$ and $d$ are shown in Fig. 1. The two $X$ actuators have masses $m_1$ and $m_2$, the cross-arm $m_b$ and the payload $m_h$. The yaw inertias of the cross-arm and payload are $J_b$ and $J_h$. The actuator damping coefficients are $c_{g1}$, $c_{g2}$ and $c_y$, while the flexible joints have stiffness $k_{b1}$, $k_{b2}$ and damping $c_{b1}$, $c_{b2}$. The distance between the centres of mass of the payload and cross-arm is $d$ [6]. The nominal values are taken from [6, Table I] and listed in Appendix A.1.

### B. Baseline state-space model

Moving the payload position $Y$ changes the mass distribution of the gantry and therefore its inertia matrix $M(Y)$. The inertia matrix (5a) is a polynomial in $Y$,

$$M(Y) = M_0 + Y M_1 + Y^2 M_2, \tag{6}$$

where $M_0$, $M_1$ and $M_2$ are given in Appendix A.2. With positive masses and inertias, $M(Y) \succ 0$ for every $Y \in \mathbb{R}$ (Appendix A.3), we invert $M(Y)$ to calculate $\ddot{q}$ from (4).

The augmentation of Section II assumes a baseline model in state-space form [5]. With the state $\tilde{x} = [q^\top\ \dot{q}^\top]^\top$, writing (4) in state-space form gives the state derivative $F$

$$\dot{\tilde{x}} = F(\tilde{x}, u) = \begin{bmatrix} \dot{q} \\ M(Y)^{-1}\big(u - C\dot{q} - Kq\big) \end{bmatrix}. \tag{7}$$

This position dependence is included by representing (7) in linear parameter-varying (LPV) form, with the payload position $Y$ as scheduling variable [16]. For a given $Y$, (7) is then linear in $\tilde{x}$ and $u$. The payload position $Y$ is both a state and the scheduling variable, making the baseline a self-scheduled LPV model [17]. Using the state $Y$ as the scheduling variable, the LPV form gives the same equations of motion as (4) for any time-varying $Y$, without making an assumption on $\dot{Y}$.

Because of $M(Y)^{-1}$ and (6), the matrices in (7) depend rationally on $Y$. We can also write the baseline as an exact linear fractional representation (LFR), with a linear time-invariant part interconnected with a scheduling block that depends on $Y$. Appendix A.2 gives the derivation and shows that the algebraic loop is well posed if $M(Y)$ is nonsingular.

### C. Physical parameters and identifiable combinations

The physical parameters are estimated jointly with the augmentation, and we require that their identifiable combinations are estimated uniquely (Section II). To prevent the definition of $q$ from being parameter-dependent, the cross-arm length $L_b$ is fixed at the nominal value listed in Appendix A.1 rather than treated as a free model parameter. The fourteen remaining parameters of Table VI in Appendix A.1, defined as $\theta$, cannot all be identified from the forces and positions in $\mathcal{D}_N$ (Section II). Two parameter sets $\theta$ and $\theta'$ that produce the same positions $q$ for every input force also have the same $\dot{q}$ and $\ddot{q}$, since both are derivatives of $q$. Writing (4) for both sets and subtracting gives

$$\big(M(Y) - M'(Y)\big)\ddot{q} + (C - C')\dot{q} + (K - K')q = 0$$

where $M(Y)$, $C$ and $K$ are evaluated at $\theta$ and $M'(Y)$, $C'$ and $K'$ at $\theta'$. With unconstrained initial conditions and full actuation ($P$ invertible), any acceleration $\ddot{q}$ can be reached by (7) at any state. Therefore the identity must hold for all these states and accelerations, which implies that $K = K'$, $C = C'$ and $M(Y) = M'(Y)$. Since $M(Y)$ is quadratic in $Y$, this also implies that $M_0 = M_0'$, $M_1 = M_1'$ and $M_2 = M_2'$. If these matrices are equal, the input-output behaviour is also the same. Since the matrices depend on ten independent combinations of $\theta$ (Appendix A.4), only these combinations can be identified. Following the idea of base parameters used in robot dynamics [18], we define them as

$$\theta_{\text{base}} = [k_{b,\Sigma}, c_{g1}, c_{g2}, c_y, c_{b,\Sigma}, m_h, m_\Sigma, m_\Delta, J_{\text{eff}}, d]^\top, \quad (8)$$

with $k_{b,\Sigma} = k_{b1} + k_{b2}$, $c_{b,\Sigma} = c_{b1} + c_{b2}$, $m_\Sigma = m_1 + m_2 + m_b$, $m_\Delta = m_1 - m_2$ and $J_{\text{eff}} = J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2)$. Since $\theta_{\text{base}}$ can be uniquely determined from the matrices (Appendix A.4), we estimate $\theta_{\text{base}}$ instead of $\theta$.

In Section III-B, $M(Y) \succ 0$ follows from positive masses and inertias. For $m_h > 0$, $m_\Sigma > 0$ and $J_{\text{eff}} > 0$, $M(Y) \succ 0$ for every $Y$ if and only if

$$m_\Sigma J_{\text{eff}} > \tfrac{L_b^2}{4} m_\Delta^2 \quad (9)$$

(Appendix A.3). To ensure that (9) holds for every estimate of $\theta_{\text{base}}$, we use a logarithmic parametrisation for the nine positive combinations and a tanh-bounded $m_\Delta$. We estimate $\theta_{\text{base}}$ through coordinates $\beta \in \mathbb{R}^{10}$, which are zero at the initial estimate $\theta^0_{\text{base}}$. The nine positive parameter combinations are $\theta_{\text{base},j} = \theta^0_{\text{base},j} \exp(\beta_j)$ for each positive $\theta_{\text{base},j}$ in (8), while $m_\Delta$ can be positive or negative and is bounded by (9):

$$m_\Delta = \gamma \frac{2}{L_b} \sqrt{m_\Sigma J_{\text{eff}}} \tanh\big(\eta_0 + s\,\beta_{m_\Delta}\big), \quad (10)$$

where $\beta_{m_\Delta}$ is the coordinate of $m_\Delta$ and $\gamma \in (0, 1)$ provides a safety margin. We choose the offset $\eta_0$ and scale $s$ so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ (Appendix A.3).

## IV. Dynamic Augmentation of the Physics Baseline

In this section, the baseline model of Section III is augmented to represent the missing dynamics introduced in Section II. We define the augmented model, simulate it inside the known feedback loop, estimate it jointly with $\theta_{\text{base}}$, and start the training from the baseline with a data-driven initialisation for the added states.

### A. Augmented model

A dynamic augmentation adds states $\bar{x}$ to model the missing dynamics, which require states the baseline does not include (Section I). The augmented model updates these states using the baseline states, the added states and the actuator forces [5, Sec. 2, Table 1].

We use the dynamic parallel structure of Hoekstra et al. [5, Table 1]. We make use of a feedforward neural network to model both the changes for the baseline states and the additional states. Fig. 3 shows how the network is connected in parallel with the baseline model. The model is defined in discrete time with the augmentation added to the state update of the baseline. This gives us

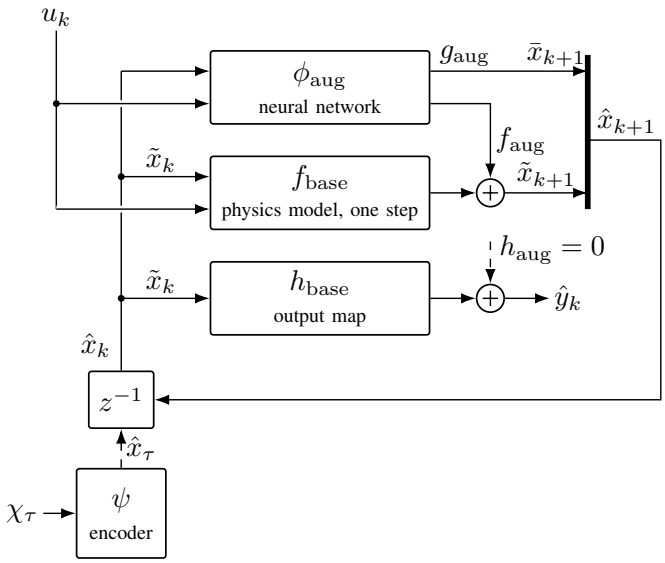



Fig. 3: Augmented model. The network $\phi_{\text{aug}}$ is connected in parallel with the baseline state-transition function $f_{\text{base}}$, which is one RK4 step of the physics model over $T_s$. Both $f_{\text{base}}$ and $h_{\text{base}}$ use the physical states $\tilde{x}_k$ of $\hat{x}_k$. The encoder $\psi$ estimates the initial state $\hat{x}_\tau$ for each simulation window.

$$\begin{aligned} \tilde{x}_{k+1} &= f_{\text{base}}(\theta_{\text{base}}, \tilde{x}_k, u_{\text{act},k}) + f_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}), \\ \bar{x}_{k+1} &= g_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}), \\ \hat{y}_k &= P^\top q_k, \end{aligned} \quad (11)$$

where $\tilde{x}_k = [q_k^\top \ \dot{q}_k^\top]^\top \in \mathbb{R}^6$ is the physical state of (7)in generalised coordinates, $\bar{x}_k \in \mathbb{R}^{n_{\bar{x}}}$ the added state, the augmented state $\hat{x}_k = [\tilde{x}_k^\top \ \bar{x}_k^\top]^\top$, $f_{\text{base}}$ and $h_{\text{base}}(\tilde{x}_k) = P^\top q_k$ the baseline state-transition and output functions, $u_{\text{act},k} \in \mathbb{R}^3$ the actuator forces, $\hat{y}_k \in \mathbb{R}^3$ the predicted actuator positions, $\theta_{\text{base}} \in \mathbb{R}^{10}$ the combinations (8), and $\theta_{\text{aug}} \in \mathbb{R}^{n_{\text{aug}}}$ the network parameters. The added states have no predetermined physical meaning, in contrast to the baseline states $\tilde{x}$, which are positions and velocities.

The dynamic-parallel augmentation structure is chosen, because the orthogonal-by-construction augmentation is defined for this additive structure [8].

The output map is not augmented ($h_{\text{aug}} = 0$ in Fig. 3), because (3) gives the exact relation between the states and the measured outputs, and this ensures that $q$ still represents the physical positions.

The baseline state-transition function $f_{\text{base}}$ (Fig. 3) is calculated using one fourth-order Runge-Kutta (RK4) step of $F$ (7) over $T_s$, with the input $u = Pu_{\text{act}}$ (3) held constant (zero-order hold) during the step. For each of the four RK4 sub-steps $M(Y)$ is evaluated using the value of $Y$ from its intermediate state.

The feedforward neural network $\phi_{\text{aug}}$ is a multilayer perceptron with tanh activation functions for the hidden layers

and a linear output layer. The inputs to the neural network are the six baseline states $\tilde{x}_k$, $n_{\bar{x}}$ added states $\bar{x}_k$ and the $n_u = 3$ actuator forces $u_{\mathrm{act},k}$. The first six outputs give the corrections of the six physical states and the remaining $n_{\bar{x}}$ outputs $g_{\mathrm{aug}}$ equal the additional states $\bar{x}_{k+1}$ (Fig. 3). The positions are also corrected as in the dynamic parallel structure [5, Table 1], because within one sample period an unmodelled force would change both the velocity and the position. The payload position $Y$ in $\tilde{x}_k$ enables the neural network to learn position-dependent effects.

The inputs to the neural network are normalised to zero mean and unit variance [19], as in [5, Sec. 5.3]. The means $\mu$ and standard deviations $\sigma$ are computed from $\mathcal{D}_N$. The baseline states are in generalised coordinates, so the measured positions are converted to $q = P^{-\top} y$, for which the velocities can be estimated from their finite difference by dividing with $T_s$. This ensures that the positions and outputs are scaled with the same measurements. The normalisation is

$$\tilde{x}^{\mathrm{n}} = T_x(\tilde{x} - \mu_x), \quad u^{\mathrm{n}} = T_u(u_{\mathrm{act}} - \mu_u), \quad y^{\mathrm{n}} = T_y(y - \mu_y), \tag{12}$$

where $\mu_x \in \mathbb{R}^6$ and $\mu_u, \mu_y \in \mathbb{R}^3$ are the sample means of $\tilde{x}$, $u_{\mathrm{act}}$ and $y$, and $T_i = \mathrm{diag}(\sigma_i)^{-1}$ for $i \in \{x, u, y\}$, with $\sigma_i$ the corresponding standard deviations. RK4 evaluates $F$ with the states and inputs in SI units, which follow from $\tilde{x}^{\mathrm{n}}$ and $u^{\mathrm{n}}$ by inverting (12). We normalise only the baseline states, since the added states $\bar{x}$ have no physical units. The scale of $\bar{x}$ is set by the network initialisation and its normalised inputs. The augmented state is then $\hat{x}^{\mathrm{n}} = [\tilde{x}^{\mathrm{n}\top}\ \bar{x}^{\top}]^{\top}$.

After normalisation the model equations are

$$\begin{aligned}
\tilde{x}^{\mathrm{n}}_{k+1} &= f^{\mathrm{n}}_{\mathrm{base}}(\theta_{\mathrm{base}}, \tilde{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k) \\
&\quad + f^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k), \\
\bar{x}_{k+1} &= g^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k), \\
\hat{y}^{\mathrm{n}}_k &= C^{\mathrm{n}} \tilde{x}^{\mathrm{n}}_k, \qquad C^{\mathrm{n}} = T_y P^{\top} [I_3 \ \ 0]\, T_x^{-1}, \\
f^{\mathrm{n}}_{\mathrm{base}}(\theta_{\mathrm{base}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) &= T_x\big(f_{\mathrm{base}}(\theta_{\mathrm{base}}, \tilde{x}, u_{\mathrm{act}}) - \mu_x\big), \\
\tilde{x} &= T_x^{-1}\tilde{x}^{\mathrm{n}} + \mu_x, \qquad u_{\mathrm{act}} = T_u^{-1} u^{\mathrm{n}} + \mu_u.
\end{aligned} \tag{13}$$

We scale $f_{\mathrm{aug}}$ only by $T_x$, since $f^{\mathrm{n}}_{\mathrm{base}}$ already subtracts $\mu_x$. We set the position means in $\mu_x$ to $P^{-\top}\mu_y$, so $\mu_y$ cancels in $T_y(P^{\top} q - \mu_y)$ and the output map remains linear without an offset. We include the means in $f^{\mathrm{n}}_{\mathrm{base}}$, whereas the transformation in [5, Sec. 5.3] assumes zero-mean signals.

The network $\phi_{\mathrm{aug}}$ uses the normalised signals:

$$\big[f^{\mathrm{n}\top}_{\mathrm{aug}}\ \ g^{\mathrm{n}\top}_{\mathrm{aug}}\big]^{\top} = \phi_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k).$$

### B. Closed-loop simulation

The model is simulated in closed-loop with the controller $K_{\mathrm{fb}}$ of (2), as in Kessels [4, Eqs. (5.12), (5.13)], instead of using the recorded input as in [5, Eq. (22)]. The reason is that the open-loop model drifts. The gantry baseline model has two axes, $X$ and $Y$, without stiffness to ground in (5), which are also affected by the learned correction. The linearised RK4 step has a pole at $z = 1$ on each of these axes, so it is not asymptotically stable. Therefore these axes act as integrators: a constant error in the correction keeps accumulating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. When the model is driven by the recorded input, nothing corrects this drift. The feedback controller is able to correct this drift.

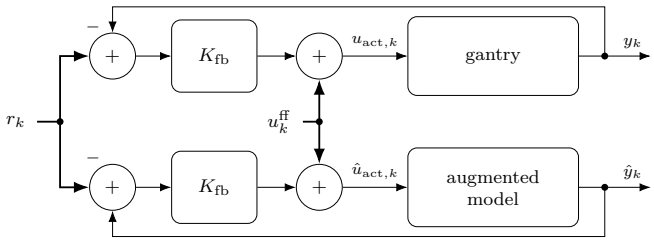



Fig. 4: Closed-loop system (top) and closed-loop model (bottom). Both use the same controller $K_{\mathrm{fb}}$, reference $r_k$ and feedforward $u^{\mathrm{ff}}_k$. In the model, the controller uses the model output $\hat{y}_k$.

The system and model use the same reference $r$ and feedforward $u^{\mathrm{ff}}$, but use their own output for feedback, with controller states $x^{\mathrm{fb}}$ and $\hat{x}^{\mathrm{fb}}$ (Fig. 4). The closed-loop system is given by (1) and (2). In the closed-loop model, the controller uses the model output $\hat{y}_k$:

$$\begin{aligned}
\hat{x}^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - \hat{y}_k), \\
\hat{u}_{\mathrm{act},k} &= C_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k,
\end{aligned} \tag{14}$$

The model uses the calculated actuator force $\hat{u}_{\mathrm{act},k}$ instead of the recorded $u_{\mathrm{act},k}$ in (11).

Since we use a linear controller we are able to subtract (2) from (14) to cancel $r$ and $u^{\mathrm{ff}}$. This leaves us with the controller state difference $x^c = \hat{x}^{\mathrm{fb}} - x^{\mathrm{fb}}$, driven by $y - \hat{y}$. The model is still simulated with respect to the reference $r$, but $r$ and $u^{\mathrm{ff}}$ are not used in simulation.

At the start of a simulation at sample $\tau$ we set $x^c_\tau = 0$. This sets the model's controller state equal to that of the system at $\tau$, because $x^c$ is the difference between the two controller states. Therefore the encoder of Section II only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [4, Remark 5.4]. We then write the closed-loop model in (14) as

$$\begin{aligned}
x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}(y_k - \hat{y}_k), \qquad x^c_\tau = 0, \\
\hat{u}_{\mathrm{act},k} &= u_{\mathrm{act},k} + C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}(y_k - \hat{y}_k),
\end{aligned} \tag{15}$$

which only needs the recorded $u_{\mathrm{act}}$ and $y$. Let $\check{u}_{\mathrm{act},k}$ be the force that $K_{\mathrm{fb}}$ computes from the recorded $r_k$ and $y_k$, with state $\check{x}^{\mathrm{fb}}_k$, plus $u^{\mathrm{ff}}_k$. With $\hat{x}^{\mathrm{fb}}_k = \check{x}^{\mathrm{fb}}_k + x^c_k$, (15) gives

$$\hat{u}_{\mathrm{act},k} = C_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k + \big(u_{\mathrm{act},k} - \check{u}_{\mathrm{act},k}\big). \tag{16}$$

The first three terms are the closed-loop model. The last term is the difference between the forces from the system and model

controllers. The noise $v$ is independent of $r$ and $u^{\mathrm{ff}}$ (Section II). If the controllers between the system and simulation differ, the equations are only approximate and the noise enters through this last term. The noise also enters through the initial state at $\tau$. The encoder computes it from the recorded $u_{\mathrm{act}}$ and $y$, and the controller state at $\tau$ depends on earlier measurements $y$. The model output $\hat{y}_k$ depends only on the state $\tilde{x}_k$, not on the input $\hat{u}_{\mathrm{act},k}$ at the same sample. For each sample we first calculate the output, then the error and the model input, before updating the states. This avoids an algebraic loop through $D_{\mathrm{fb}}$, for the same reason as Kessels [4, Remark 5.1]. For a simulation starting at $\tau$, the model equations in the normalised coordinates of (12) are

$$\begin{aligned} \hat{y}^{\mathrm{n}}_{k|\tau} &= C^{\mathrm{n}}\tilde{x}^{\mathrm{n}}_{k|\tau}, \qquad \varepsilon_{k|\tau} = T_y^{-1}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big), \\ \hat{u}^{\mathrm{n}}_{k|\tau} &= u^{\mathrm{n}}_k + T_u\big(C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}\varepsilon_{k|\tau}\big), \\ x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}\varepsilon_{k|\tau}, \qquad x^c_\tau = 0, \end{aligned} \tag{17}$$

and the next model state $\hat{x}^{\mathrm{n}}_{k+1|\tau}$ is calculated using (13) with input $\hat{u}^{\mathrm{n}}_{k|\tau}$.

### C. Identification

This subsection states how the augmented model is estimated from $\mathcal{D}_N$. We first motivate the encoder, then give the cost function, the encoder and the optimiser.

The data $\mathcal{D}_N$ consist of multiple records. Each simulation window starts at a sample $\tau$ from an initial state $\hat{x}_\tau$, but we measure only the positions of the physical state $\tilde{x}$. The additional states $\bar{x}$ are latent, but we need their initial values at $\tau$ to simulate each window. Within the window, $k$ denotes the sample. Estimating $\hat{x}_\tau$ as a separate parameter for each window would increase the number of parameters as the data grows [15, Sec. 3.1]. Following Beintema et al., we use an encoder network that estimates $\hat{x}_\tau$ from the past inputs and outputs $\chi_\tau$ [15, Sec. 3.2]. This keeps the computation per gradient step independent of the record length, for a fixed batch size and window length [15, Sec. 3.1, Eq. (9)]. Using short simulation windows increases the loss function smoothness [15, Sec. 3.1, 4.2].

The network parameters $\theta_{\mathrm{aug}}$, the encoder parameters $\theta_{\mathrm{enc}}$ and $\theta_{\mathrm{base}}$ are jointly estimated, as in [5, Sec. 5.1], while the encoder paper keeps $\theta_{\mathrm{base}}$ fixed [21, footnote 1]. The parameters $\vartheta = (\beta, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the known controller $K_{\mathrm{fb}}$. We adapt this from [5, Eq. (22)] for the closed loop of Section IV-B and the encoder:

$$\begin{aligned} &\min_{\vartheta}\ V(\vartheta) = \frac{1}{|\mathcal{W}|\, n_f\, n_y} \sum_{\tau\in\mathcal{W}} \sum_{k=\tau}^{\tau+n_f-1} \Big\| W_r\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big)\Big\|_2^2 \quad \text{s.t.} \\ &\tilde{x}^{\mathrm{n}}_{k+1|\tau} = f^{\mathrm{n}}_{\mathrm{base}}\big(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big) + f^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big), \\ &\bar{x}_{k+1|\tau} = g^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big), \\ &\hat{x}^{\mathrm{n}}_{\tau|\tau} = \psi(\theta_{\mathrm{enc}}, \chi_\tau), \quad \hat{y}^{\mathrm{n}}_{k|\tau},\ \hat{u}^{\mathrm{n}}_{k|\tau} \text{ from (17)}, \end{aligned} \tag{18}$$

for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$ of record $r$ and has length $n_f$, $n_y = 3$, and $\theta_{\mathrm{base}}(\beta)$ gives the baseline parameters (Section III-C). $W_r = \mathrm{diag}(\sigma_{y,j}/\sigma_{e,r,j})$ weights axis $j$ of record $r$, so that its residual in metres is divided by the standard deviation $\sigma_{e,r,j}$ of its servo error $y_j - r_j$.

In (18), the encoder $\psi$ uses the samples up to $\tau - 1$, as in [5, Eq. (22e)]:

$$\begin{aligned} \hat{x}^{\mathrm{n}}_{\tau|\tau} &= \psi(\theta_{\mathrm{enc}}, \chi_\tau), \\ \chi_\tau &= \big[\,(y^{\mathrm{n}}_{\tau-n-1})^\top \ \cdots\ (y^{\mathrm{n}}_{\tau-1})^\top \\ &\qquad (u^{\mathrm{n}}_{\tau-n-1})^\top \ \cdots\ (u^{\mathrm{n}}_{\tau-1})^\top\big]^\top, \end{aligned} \tag{19}$$

where $\hat{x}^{\mathrm{n}}_{\tau|\tau}$ is the normalised state $\hat{x}_\tau$ of Fig. 3 at the start of the window, $n+1$ is the number of past samples used by the encoder, and $\chi_\tau \in \mathbb{R}^{6(n+1)}$ stacks the normalised recorded outputs and inputs as in (12). The encoder estimates all $6 + n_{\bar{x}}$ initial states $\hat{x}_\tau$, which includes the measured positions. We estimate the positions instead of using the measured $y_\tau$ to keep the positions consistent with the estimated velocities.

The encoder uses a linear map with a nonlinear correction [21, Eq. (8)]:

$$\psi(\theta_{\mathrm{enc}}, \chi_\tau) = \begin{bmatrix} W^b \\ W^a \end{bmatrix} \chi_\tau + \tilde{\psi}(\chi_\tau), \tag{20}$$

where $W^b \in \mathbb{R}^{6\times 6(n+1)}$ and $W^a \in \mathbb{R}^{n_{\bar{x}}\times 6(n+1)}$ are the linear maps for the six physical states $\psi_b$ and the $n_{\bar{x}}$ additional states $\psi_a$, and $\tilde{\psi}$ is a neural network. The encoder parameters $\theta_{\mathrm{enc}}$ are $W^b$, $W^a$ and the parameters of $\tilde{\psi}$.

SUBNET assumes that the input is independent of the noise [15, Sec. 2.1] and that the system that generates the data is incrementally exponentially output stable [15, Cond. 1]. The identification method we have adapted from [5] relies on these SUBNET assumptions for its convergence and consistency guarantees [5, Sec. 5.5]. The open-loop gantry system does not satisfy the second: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset.

Each window starts from a new encoder estimate and $x^c_\tau = 0$, so the window is not affected by model errors before $\tau$.

The baseline parameters and the neural network can both correct the same model errors, making the optimal parameter values not unique [5, Sec. 5.2].

For training we minimise $V$ in (18) with Adam. Each update uses a minibatch of windows, and the gradient is calculated over the full window. The parameters are selected on a separate validation set $\mathcal{D}_{\mathrm{val}}$. The records of $\mathcal{D}_{\mathrm{val}}$ are simulated in closed-loop and only initialised at the start. The encoder uses the first $n+1$ samples for this, and the validation error is calculated over the remaining samples. During the optimisation with Adam the parameters with the lowest validation error are kept. From these parameters, we continue to minimise $V$ with L-BFGS on a fixed set of windows. We call this step polish, and the parameter updates from this step are only kept if they lower the validation error. Drenth et al. also use Adam followed by L-BFGS-B [17, Sec. 4.1]. Because every $\beta$ satisfies (9), unconstrained L-BFGS works without a bound.

### D. Initialisation

A fully random initialisation can make the model unstable and lead to poor optimisation results. We therefore start training from a model that behaves as the baseline [5, Sec. 5.4].

The physical rows are initialised from the baseline model. The baseline is nonlinear, because solving (4) for $\ddot{q}$ requires $M(Y)^{-1}$ and $Y$ is a state (Section III-B). Hoekstra et al.'s model-based state reconstruction map for the nonlinear baseline needs a linearisation point, so the map is only accurate around that point [21, Sec. 3.1]. Hoekstra et al. therefore suggest fitting the encoder to states simulated with the baseline [21, Sec. 3.2, Eq. (35)], as their augmentation method does [5, Sec. 5.4.2]. We apply this approach and simulate the baseline model once before training to obtain the physical state at every sample $k$. The recorded actuator forces $u_{\text{act}}$ and positions $y$ from the records of $\mathcal{D}_N$ with the known controller in (17) are used to simulate the physical states in closed-loop:

$$\tilde{x}^{\text{n},0}_{k+1} = f^{\text{n}}_{\text{base}}\big(\theta^0_{\text{base}}, \tilde{x}^{\text{n},0}_k, \hat{u}^{\text{n},0}_k\big), \qquad \hat{y}^{\text{n},0}_k = C^{\text{n}}\tilde{x}^{\text{n},0}_k, \quad (21)$$

where $\tilde{x}^{\text{n},0}_k$ is the simulated baseline state, $\theta^0_{\text{base}}$ are the initial parameters, and $\hat{u}^{\text{n},0}_k$ is the closed-loop input from (17). We use the closed-loop input from (21), instead of the recorded input as in [5, Sec. 5.3]. We use $\hat{y}^{\text{n},0}_k$ as the model output. For our closed-loop simulation, each record starts from the measured positions $q = P^{-\top}y$ in generalised coordinates. The velocities, the added states and $x^c$ start at zero. The physical rows of the encoder are then fitted to the simulated states by least squares [21, Eq. (33)]:

$$\big(W^{b,0}, b^{b,0}\big) \in \arg\min_{W^b,\, b} \frac{1}{|\mathcal{D}|}\sum_{\tau\in\mathcal{D}} \big\|W^b\chi^0_\tau + b - \tilde{x}^{\text{n},0}_\tau\big\|_2^2, \quad (22)$$

where $b \in \mathbb{R}^6$ is a constant offset. $\chi^0_\tau$ is built as $\chi_\tau$ in (19), with $\hat{y}^{\text{n},0}$ and $\hat{u}^{\text{n},0}$ instead of the recorded $y^{\text{n}}$ and $u^{\text{n}}$. $\mathcal{D}$ is the set of samples $\tau$ where we fit the encoder in (22). Hoekstra et al. use this linear fit for LTI baselines and fit a neural network for nonlinear baselines [21, Eqs. (33), (35)]. We only fit the linear map $W^b$. The nonlinear correction $\tilde{\psi}$ is learned during training. The output bias of the physical rows of $\tilde{\psi}$ starts at the offset $b^{b,0}$, and their output weights start at zero. This gives $\psi_b = W^{b,0}\chi_\tau + b^{b,0}$ at the start.

To start from the baseline, we zero the physical-state rows of the output layer of $\phi_{\text{aug}}$ and keep the added-state rows random:

$$\begin{aligned}
&\text{baseline:} && \beta = 0,\\
&\text{network:} && W^{\bar{x}}_L = 0, \quad b_L = 0, \quad W^{\bar{x}}_L \sim \mathcal{U}\big(-1/\sqrt{n_h}, 1/\sqrt{n_h}\big),\\
&\text{encoder:} && W^b = W^{b,0}, \quad \tilde{W}^{\tilde{x}}_L = 0, \quad \tilde{b}^{\tilde{x}}_L = b^{b,0},\\
& && W^a \sim \mathcal{U}(-a, a), \quad \tilde{W}^{\bar{x}}_L = 0, \quad \tilde{b}^{\bar{x}}_L = 0,\\
& && a = \sqrt{6/(n_{\bar{x}} + 6(n+1))},
\end{aligned} \quad (23)$$

where $W_L$ and $b_L$ are the output layer weights and bias of $\phi_{\text{aug}}$. The rows $W^{\tilde{x}}_L$ and $W^{\bar{x}}_L$ are for $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$, respectively. $n_h$ is the number of neurons in each hidden layer. For $\tilde{\psi}$, the output layer weights and bias are split into $\tilde{W}^{\tilde{x}}_L$, $\tilde{b}^{\tilde{x}}_L$ for the physical states and $\tilde{W}^{\bar{x}}_L$, $\tilde{b}^{\bar{x}}_L$ for the added states. Here $W \sim \mathcal{U}(-a, a)$ means each entry of $W$ is drawn independently from a uniform distribution on $[-a, a]$. We initialise $W^{\bar{x}}_L$ and the hidden layers of $\phi_{\text{aug}}$ and $\tilde{\psi}$ from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where $m$ is the input size of the layer. We use Xavier initialisation for $W^a$, following [5, Eq. (31)]. We choose the uniform distribution with gain one. At the start of training, $\theta_{\text{base}} = \theta^0_{\text{base}}$ and $f^{\text{n}}_{\text{aug}} = 0$. The augmented model and baseline model give the same $\tilde{x}$ and $\hat{y}$, given they start from the same physical state and receive the same input. The added states $\bar{x}$ are still updated by $g^{\text{n}}_{\text{aug}}$, as in [5, Eq. (29)].

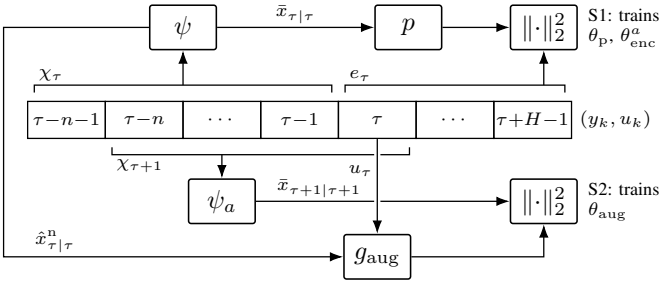



Fig. 5: Samples used by PS on one record, with the layout adapted from [22, Fig. 2]. In S1 (top), the encoder $\psi$ estimates $\hat{x}^{\text{n}}_{\tau|\tau}$ from the window $\chi_\tau$. The decoder $p$ then predicts the residual $e_\tau$ of the next $H$ samples from the added states $\bar{x}_{\tau|\tau}$ (24). In S2 (bottom), $g_{\text{aug}}$ predicts the next added state $\bar{x}_{\tau+1|\tau+1}$ from $\hat{x}^{\text{n}}_{\tau|\tau}$ and the recorded $u_\tau$ (26). The same encoder gives $\bar{x}_{\tau+1|\tau+1}$ from the window $\chi_{\tau+1}$, shifted by one sample.

### E. Added-state initialisation

The physical states have a data-based start (22) and a direct map to the output, while the added states have neither. The added states start from the random rows $W^a$ and are updated through the random added-state rows of $\phi_{\text{aug}}$. They enter $\tilde{x}_{k+1}$ only through $f^{\text{n}}_{\text{aug}}$ in (18). $V$ has no term that requires $\bar{x}$ to predict the output error, so $f^{\text{n}}_{\text{aug}}$ can lower part of this error with $\tilde{x}$ and $\hat{u}$ alone. Minimising $V$ can therefore reduce the error without capturing the missing dynamics in $\bar{x}$.

We therefore propose predictive-state (PS) initialisation, which is a data-driven initialisation for the added states. PS has two stages, each with its own cost function. In the first stage (S1), the encoder's added-state outputs are trained to predict the residual of the augmented model. In the second stage (S2), $g^{\text{n}}_{\text{aug}}$ is trained to predict the next added state. After S2, we continue minimising $V$. PS is adapted from a two-stage regression of predictive-state learning [22, Sec. 2], which Downey et al. use to initialise a recurrent network before backpropagation through time (BPTT) [23, Sec. 4.2]. S1 predicts the residual of the model, instead of features of future observations as in [22]. S2 trains the nonlinear state update instead of a linear map. We continue minimising $V$ during the first stage. Fig. 5 shows which samples are used for each stage.

In S1, the residual is computed with the model parameters $\vartheta$ from the latest update of $V$. The encoder parameters for

the added states are $W^a$ and the added-state rows $\tilde{W}_L^{\bar{x}}$, $\tilde{b}_L^{\bar{x}}$ of the output layer of $\tilde{\psi}$. We call these $\theta_{\text{enc}}^a$. For the S1 update we select a minibatch $\mathcal{B}$ of samples $\tau$ from $\mathcal{D}_N$. A subset of records is left out of S1 and S2 to evaluate the performance of each stage. From the encoder state $\hat{x}^{\text{n}}_{\tau|\tau}$ the augmented model is simulated in the closed loop of Section IV-B. We calculate the residual $y - \hat{y}$ over the next $H = n + 1$ samples, from $\tau$ to $\tau + H - 1$. The top of Fig. 5 shows the window $\chi_\tau$ and the samples of the residual $e_\tau$. Then a temporary decoder $p$, a neural network with one hidden layer, with only the added states as inputs $\bar{x}_{\tau|\tau} = \psi_a(\theta_{\text{enc}}, \chi_\tau)$ is used to predict this residual:

$$\min_{\theta_{\text{p}},\, \theta_{\text{enc}}^a} \frac{1}{|\mathcal{B}|\, H\, n_y} \sum_{\tau \in \mathcal{B}} \left\| p\big(\theta_{\text{p}}, \psi_a(\theta_{\text{enc}}, \chi_\tau)\big) - e_\tau \right\|_2^2,$$
$$e_\tau = \frac{1}{\sigma_e} \left[ (y^{\text{n}}_\tau - \hat{y}^{\text{n}}_{\tau|\tau})^\top \ \cdots \ (y^{\text{n}}_{\tau+H-1} - \hat{y}^{\text{n}}_{\tau+H-1|\tau})^\top \right]^\top, \tag{24}$$

where $\hat{y}^{\text{n}}_{k|\tau}$ is the output from the closed-loop simulation of Section IV-B, starting at $\hat{x}^{\text{n}}_{\tau|\tau}$. This simulated output is kept fixed during the S1 update. $\sigma_e$ is the RMS of all residual values in $\mathcal{B}$. Only $\theta_{\text{p}}$ and $\theta_{\text{enc}}^a$ are updated by S1. The parameters of $\vartheta$, including $\theta_{\text{enc}}^a$ are still trained through $V$.

The expected squared error is minimised by the conditional mean $\mathbb{E}[e_\tau \mid \chi_\tau]$. The decoder only sees up to $\bar{x}_{\tau|\tau}$, not the full history $\chi_\tau$, this encourages the encoder to use the added states to predict the future residual.

S1 ends when the decoder's prediction error on the left out records stops improving, and S2 runs only if the decoder explains at least 10 % of the residual there. The left out records are still used to minimise $V$. During S1, each Adam step on $V$ is followed by one Adam step on (24). For $i = 0, 1, \ldots, i_1 - 1$ this gives

$$\begin{aligned} \vartheta^{(i+1)} &= \mathcal{A}_V\big(\vartheta^{(i)}\big), \\ \big(\theta_{\text{p}}^{(i+1)}, \theta_{\text{enc}}^{a,(i+1)}\big) &= \mathcal{A}_{\text{S1}}\big(\theta_{\text{p}}^{(i)}, \theta_{\text{enc}}^a \text{ of } \vartheta^{(i+1)}\big), \end{aligned} \tag{25}$$

where $\mathcal{A}_V$ and $\mathcal{A}_{\text{S1}}$ are one Adam step on (18) and (24), each with its own Adam optimiser. Both steps update $\theta_{\text{enc}}^a$, so the $\theta_{\text{enc}}^a$ from $\mathcal{A}_{\text{S1}}$ replaces the one in $\vartheta^{(i+1)}$, and $i_1$ is the last update of S1.

S1 has only trained the part of the encoder that initialises the added states $\bar{x}_{\tau|\tau}$. S2 trains the state update $g^{\text{n}}_{\text{aug}}$ itself, to predict the next added state $\bar{x}_{k+1}$ from the full state $\hat{x}^{\text{n}}_k$ and the input $u^{\text{n}}_k$. For S2 we select samples $k_i$ from $\mathcal{D}_N$ without the left out records. For each sample the encoder estimates $\hat{x}^{\text{n}}_{k_i|k_i}$ from $\chi_{k_i}$ and $\bar{x}_{k_i+1|k_i+1}$ from the next window $\chi_{k_i+1}$. In Fig. 5, $k_i$ is the sample $\tau$. We calculate $\hat{x}^{\text{n}}_{k_i|k_i}$ and $\bar{x}_{k_i+1|k_i+1}$ once after S1. Together with the recorded inputs $u^{\text{n}}_{k_i}$, they form the fixed set $\mathcal{P}$ in (26).

During S2 the model is not simulated in closed-loop and $V$ is not minimised. We instead train $g^{\text{n}}_{\text{aug}}$ on $\mathcal{P}$ using one-step regression with the recorded actuator force $u^{\text{n}}_{k_i}$ as input, and a separate Adam optimiser. At the start of a window, the recorded force equals the closed-loop input of $V$ when the encoder reproduces $y_{k_i}$, by (15) with $x^c_\tau = 0$. S2 solves

$$\begin{aligned} \mathcal{P} &= \big\{(\hat{x}^{\text{n}}_{k_i|k_i},\, u^{\text{n}}_{k_i},\, \bar{x}_{k_i+1|k_i+1})\big\}_{i=1}^{N_\mathcal{P}}, \\ \hat{x}^{\text{n}}_{k_i|k_i} &= \psi(\theta_{\text{enc}}, \chi_{k_i}), \quad \bar{x}_{k_i+1|k_i+1} = \psi_a(\theta_{\text{enc}}, \chi_{k_i+1}), \\ \min_{\theta_{\text{aug}}} & \frac{\sum_{i \in \mathcal{B}_\mathcal{P}} \left\| g^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k_i|k_i}, u^{\text{n}}_{k_i}) - \bar{x}_{k_i+1|k_i+1} \right\|_2^2}{\sum_{i \in \mathcal{B}_\mathcal{P}} \sum_{j=1}^{n_{\bar{x}}} \big(\bar{x}_{j,k_i+1|k_i+1} - m_j\big)^2}, \end{aligned} \tag{26}$$

where $\theta_{\text{enc}}$ is not updated during S2, $\mathcal{B}_\mathcal{P} \subset \mathcal{P}$ is a minibatch, $\bar{x}_{j,k_i+1|k_i+1}$ is the $j$th added state and $m_j$ is the mean of $\bar{x}_{j,k_i+1|k_i+1}$ over $\mathcal{B}_\mathcal{P}$. We divide the prediction error by the variance of $\bar{x}_{k_i+1|k_i+1}$ over $\mathcal{B}_\mathcal{P}$ to keep the cost independent of the scale of $\bar{x}$.

We keep the network parameters $\theta_{\text{aug}}$ with the lowest cost (26) on the left out records. S2 updates all parameters $\theta_{\text{aug}}$ of $\phi_{\text{aug}}$. The physical rows of $W_L$ and $b_L$ stay unchanged, because (26) only depends on the added-state rows, but the shared hidden layers also change $f^{\text{n}}_{\text{aug}}$. After S2, we continue minimising (18) from

$$\vartheta_{\text{PS}} = \big(\beta^{(i_1)},\, \theta_{\text{aug}}^{\text{S2}},\, \theta_{\text{enc}}^{(i_1)}\big), \tag{27}$$

followed by the polish and the selection of Section IV-C. Here $\beta^{(i_1)}$ and $\theta_{\text{enc}}^{(i_1)}$ are the parameters after S1 and $\theta_{\text{aug}}^{\text{S2}}$ after S2. When S2 is skipped, $\vartheta_{\text{PS}}$ contains the parameters after S1. PS only changes the starting point of minimising $V$, not what is minimised. PS adds no parameters to the augmented model, because the decoder is removed after S1.

## V. Interpretability by Orthogonal Construction

In this section, we make the estimate of $\theta_{\text{base}}$ unique. The parameter pair $(\theta_{\text{base}}, \theta_{\text{aug}})$ minimising the loss function of Section IV-C can be non-unique: different pairs can give the same fit [8, Sec. 3.1]. Then $\theta_{\text{base}}$ is not interpretable as defined in Section I. To prevent the neural network from correcting what $\theta_{\text{base}}$ can represent we introduce orthogonality by construction [8]. Orthogonality here is defined as the neural network correction to the physical states having no component in the directions in which a change of $\theta_{\text{base}}$ is able to change the next state predicted by the baseline. This gives us a unique estimate of $\theta_{\text{base}}$ up to first order. This estimate equals the true parameters only if the missing dynamics themselves are orthogonal to these directions.

Györök et al. have constructed this for input-output baseline models that are linear in their parameters [8, Sec. 2]. We extend this to state-space models nonlinear in their parameters, specifically for our gantry system.

### A. Orthogonal-by-construction parametrisation

For a baseline linear in its parameters, $\hat{y}_k = \varphi(x_k)\theta_{\text{b}}$, changing $\theta_{\text{b}}$ changes the model output $\hat{y}_k$ by a linear combination of the columns of the regressor $\varphi(x_k)$ [8, Eqs. (2), (39)]. Györök

et al. subtract from the neural network output its least-squares fit by the stacked regressor [8, Eqs. (8), (9), (11), Lemma 3]:

$$\begin{aligned} \hat{Y} &= \Phi\theta_{\mathrm{b}} + \tilde{F}^{\mathrm{ANN}}, \\ \tilde{F}^{\mathrm{ANN}} &= F^{\mathrm{ANN}} - \Phi\theta_{\mathrm{aux}}, \\ \theta_{\mathrm{aux}} &= \left(\Phi^{\top}\Phi\right)^{-1}\Phi^{\top}F^{\mathrm{ANN}}, \\ 0 &= \Phi^{\top}\tilde{F}^{\mathrm{ANN}}, \end{aligned} \tag{28}$$

where $x_k$ is the vector of past measured outputs and past and current inputs [8, Eq. (1)], $\hat{Y}$ stacks the predicted outputs over $\mathcal{D}_N$, $\Phi$ is the stacked regressor, $F^{\mathrm{ANN}}$ stacks the parallel network outputs [8, Eq. (5)] and $\tilde{F}^{\mathrm{ANN}}$ is the projected network output [8, Eqs. (9), (10)].

The projected network output $\tilde{F}^{\mathrm{ANN}}$ is orthogonal to the regressor [8, Lemma 3]. Calculating $\theta_{\mathrm{aux}}$ requires $(\Phi^{\top}\Phi)^{-1}$, which exists only if $\Phi$ has full column rank [8, Assumption 1]. Therefore different values of $\theta_{\mathrm{b}}$ are unable to give the same stacked prediction on $\mathcal{D}_N$. An input with this property is weakly persistently exciting [8, Eq. (6), Sec. 4.3]. This also requires an identifiable baseline, where different values of $\theta_{\mathrm{b}}$ give different functions $\varphi(\cdot)\theta_{\mathrm{b}}$.

### B. Extension to the gantry

Our baseline violates three requirements of the projection in (28). First, its state update is nonlinear in the parameters, because the next state update depends on $M(Y)^{-1}$ in (7), violating the linear requirement for the regressor. Instead of the regressor the Jacobian of the baseline state update is taken with respect to $\beta$, as in the orthogonal regularisation method of Györök et al. [25, Sec. 4, Eq. (15)]. Like the regressor, the stacked Jacobian must have full column rank. This requires identifiable parameters, which is why we use $\beta$(Section III-C). The Jacobian is taken at a linearisation point $\beta_{\mathrm{lin}}$ as

$$\varphi(\beta_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) = \left.\frac{\partial f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right)}{\partial\beta}\right|_{\beta=\beta_{\mathrm{lin}}}, \tag{29}$$

where $\beta_{\mathrm{lin}} \in \mathbb{R}^{10}$ and $\varphi : \mathbb{R}^{10} \times \mathbb{R}^{6} \times \mathbb{R}^{3} \to \mathbb{R}^{6\times 10}$. A first-order expansion of the baseline state update gives [25, Eq. (16)]:

$$\begin{aligned} f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right) &\approx \varphi\,\beta + \gamma, \\ \gamma &= f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta_{\mathrm{lin}}), \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}\right) - \varphi\,\beta_{\mathrm{lin}}, \end{aligned} \tag{30}$$

where the first term is linear in $\beta$ and $\gamma : \mathbb{R}^{10}\times\mathbb{R}^{6}\times\mathbb{R}^{3} \to \mathbb{R}^{6}$is an offset independent of $\beta$. Györök et al. add this offset as an extra column to the Jacobian, to promote orthogonality between the neural network and the next states predicted by the baseline [25, Eq. (18)]. We include it as well.

Second, the Jacobian also depends on the state, while the regressor of Györök et al. uses only measured inputs and outputs [8, Eqs. (1), (2)]. Since only the positions are measured, we evaluate the Jacobian at these positions with the velocities reconstructed from them:

$$\begin{aligned} \tilde{x}_i &= \mathrm{col}(q_i, \dot{q}_i), \qquad q_i = P^{-\top} y_i, \\ \dot{q}_i &= \frac{-q_{i+2} + 8q_{i+1} - 8q_{i-1} + q_{i-2}}{12\,T_s}, \end{aligned} \tag{31}$$

where $y_i$ is the measured actuator position at sample $i$. Each tuple $(\tilde{x}_i, \bar{x}_i, u_i)$ uses the recorded actuator force $u_i$ and $\bar{x}_i = 0$, with the physical state and input normalised to $\tilde{x}^{\mathrm{n}}_i$ and $u^{\mathrm{n}}_i$ using (12). The tuples are calculated once before training, using all samples for which (31) can be calculated at $i$ and at the next sample $i+1$.

Stacking the Jacobian, the offset and the network output over the tuples, the projection of (28) becomes

$$\begin{aligned} \Phi_{\Gamma} &= \mathrm{col}\big([\varphi \;\; \gamma](\beta_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}_i, u^{\mathrm{n}}_i)\big)_{i\in\mathcal{I}}, \\ F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) &= \mathrm{col}\big(f^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \mathrm{col}(\tilde{x}^{\mathrm{n}}_i, 0), u^{\mathrm{n}}_i)\big)_{i\in\mathcal{I}}, \\ \theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}) &= \left(\Phi_{\Gamma}^{\top}\Phi_{\Gamma}\right)^{-1}\Phi_{\Gamma}^{\top}F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}), \\ \tilde{F}^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) &= F^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}) - \Phi_{\Gamma}\,\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}), \\ 0 &= \Phi_{\Gamma}^{\top}\tilde{F}^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}), \end{aligned} \tag{32}$$

where $\mathcal{I}$ is the set of tuples, $N = |\mathcal{I}|$, $\Phi_{\Gamma} = [\Phi\;\Gamma] \in \mathbb{R}^{6N\times 11}$ stacks $\varphi$ and $\gamma$ evaluated at $\beta_{\mathrm{lin}}$, $F^{\mathrm{n}}_{\mathrm{aug}}, \tilde{F}^{\mathrm{n}}_{\mathrm{aug}} \in \mathbb{R}^{6N}$ and $\theta_{\mathrm{aux}} \in \mathbb{R}^{11}$. The projected network output $\tilde{F}^{\mathrm{n}}_{\mathrm{aug}}$ therefore has no component along any column of $\Phi_{\Gamma}$, which satisfies our orthogonality definition at the start of this section. This holds for the stacked tuples at $\bar{x} = 0$, not for each tuple separately. Third, we use a state-space model predicting the next state instead of the output. The fitted part is removed from the neural network correction of the physical state at every step in the closed-loop simulation, adapting the output correction in [8, Eqs. (9), (13)]. At each step, we subtract the Jacobian and offset multiplied by $\theta_{\mathrm{aux}}$ from the network correction, evaluated at the current state and input. The baseline has no added states, so its Jacobian and offset are zero in the added-state rows and $g^{\mathrm{n}}_{\mathrm{aug}}$ is orthogonal to it by definition. We therefore only correct the physical rows:

$$\begin{aligned} f^{\mathrm{n}}_{\mathrm{obc}}(\theta_{\mathrm{aug}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) &= [\varphi \;\; \gamma](\beta_{\mathrm{lin}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}})\,\theta_{\mathrm{aux}}(\theta_{\mathrm{aug}}), \\ \tilde{x}^{\mathrm{n}}_{k+1|\tau} &= f^{\mathrm{n}}_{\mathrm{base}} + f^{\mathrm{n}}_{\mathrm{aug}} - f^{\mathrm{n}}_{\mathrm{obc}}, \\ \bar{x}_{k+1|\tau} &= g^{\mathrm{n}}_{\mathrm{aug}}, \end{aligned} \tag{33}$$

where $f^{\mathrm{n}}_{\mathrm{obc}} : \mathbb{R}^{n_{\mathrm{aug}}} \times \mathbb{R}^{6} \times \mathbb{R}^{3} \to \mathbb{R}^{6}$ is evaluated at $(\tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau})$, and $f^{\mathrm{n}}_{\mathrm{base}}$, $f^{\mathrm{n}}_{\mathrm{aug}}$ and $g^{\mathrm{n}}_{\mathrm{aug}}$ are evaluated as in (18). We differentiate $\theta_{\mathrm{aux}}$ with respect to $\theta_{\mathrm{aug}}$, using its expression in [8, Eq. (8)], to train the network using the gradient of its projected output.

### C. Identification and recovery

The augmented model is identified as in Section IV-C, with the added-state initialisation of Section IV-E. With orthogonality by construction the state update in (18) is instead the projected state update (33). Since $\theta_{\mathrm{aux}}$ depends on $\theta_{\mathrm{aug}}$, we solve (32) at every evaluation of $V$. At the start of each epoch $\beta_{\mathrm{lin}}$ is set to the current $\beta$, and is held fixed during the epoch. Györök et al. propose updating $\beta_{\mathrm{lin}}$ at every iteration or keeping it fixed at the nominal parameters [25, Sec. 4]. To allow training to start from detuned parameterswithout rebuilding $\Phi_{\Gamma}$ at each evaluation of $V$, we instead adapt their recomputation at the start of each epoch [25, Remark 2]. The projection does not guarantee that the true parameters $\beta^{\star} \in \mathbb{R}^{10}$ are recovered.

That depends on whether the missing dynamics on the tuples are orthogonal to the stacked Jacobian:

$$\Xi = \operatorname{col}\big(\tilde{x}^{\mathrm{n}}_{i+1} - f^{\mathrm{n}}_{\mathrm{base}}(\theta_{\mathrm{base}}(\beta^\star), \tilde{x}^{\mathrm{n}}_i, u^{\mathrm{n}}_i)\big)_{i\in\mathcal{I}}, \tag{34}$$

where $\Xi \in \mathbb{R}^{6N}$ contains the unmodelled terms of [8, Eqs. (14), (15)], the reconstruction error of (31) and the measurement noise. To first order around $\beta^\star$, the one-step error on the tuples is $\Xi - \Phi(\beta - \beta^\star) - \tilde{F}^{\mathrm{n}}_{\mathrm{aug}}$. Minimising the squared one-step error over $\beta$ gives $\hat{\beta}$. Without the projection, $F^{\mathrm{n}}_{\mathrm{aug}}$ replaces $\tilde{F}^{\mathrm{n}}_{\mathrm{aug}}$, which gives $\hat{\beta}_{\mathrm{np}}$:

$$\begin{aligned} \hat{\beta} &= \beta^\star + \big(\Phi^\top\Phi\big)^{-1}\Phi^\top\Xi, \\ \hat{\beta}_{\mathrm{np}} &= \beta^\star + \big(\Phi^\top\Phi\big)^{-1}\Phi^\top\big(\Xi - F^{\mathrm{n}}_{\mathrm{aug}}\big). \end{aligned} \tag{35}$$

With the projection, $\hat{\beta}$ does not depend on the network output, since $\Phi^\top\tilde{F}^{\mathrm{n}}_{\mathrm{aug}} = 0$ by (32). The offset does not change the first-order estimate (35). Without the projection, $\Phi^\top F^{\mathrm{n}}_{\mathrm{aug}}$ is still in the estimate $\hat{\beta}_{\mathrm{np}}$ [8, Eq. (25)]. The expression for $\hat{\beta} - \beta^\star$ has the same form as Györök et al. [8, Eq. (21)], which they derived assuming the model exactly recovers the dynamics on $\mathcal{D}_N$ [8, Assumption 6]. We obtain it by minimising the squared one-step error. The estimate equals $\beta^\star$ if and only if $\Phi^\top\Xi = 0$, which means that $\Xi$ is orthogonal to the columns of $\Phi$ on the tuples [8, Condition 4, Remark 5]. This holds for the one-step error, not for the closed-loop loss (18).

### D. Overlap with the parameter directions

We measure orthogonality using the normalised inner product, with the physical state normalised using (12):

$$\begin{aligned} \rho(Z) &= \max_{Q\in\operatorname{range}\Phi\setminus\{0\}} \frac{\langle Q, Z\rangle}{\|Q\|_2\,\|Z\|_2} = \frac{\|\Pi Z\|_2}{\|Z\|_2}, \\ \Pi &= \Phi\big(\Phi^\top\Phi\big)^{-1}\Phi^\top, \end{aligned} \tag{36}$$

where $Z \in \mathbb{R}^{6N}$ is nonzero and $\Pi$ is the orthogonal projector onto $\operatorname{range}\Phi$. The overlap $\rho(Z)$ is the largest normalised inner product between $Z$ and a vector in $\operatorname{range}\Phi$. It is zero if and only if $\Phi^\top Z = 0$.

## VI. Experiment Design

In this section, we describe the simulated system, how the data is generated, the records, the compared models and the metrics. The data used to validate our methods is simulated, with limits and noise level taken from a gantry of ASMPT. Its data is confidential, therefore positions are given relative to the training range, and velocities and accelerations relative to the machine limits. The design details are in Appendix C.

### A. Simulated system

The system used to generate the data consists of the baseline model (4) with two additions. Fig. 6 shows the baseline model with both additions. The first is Coulomb friction at the actuators $X_1$, $X_2$ and $Y$, as modelled by García-Herreros et al. [6]. We left it out of the baseline to add nonlinear

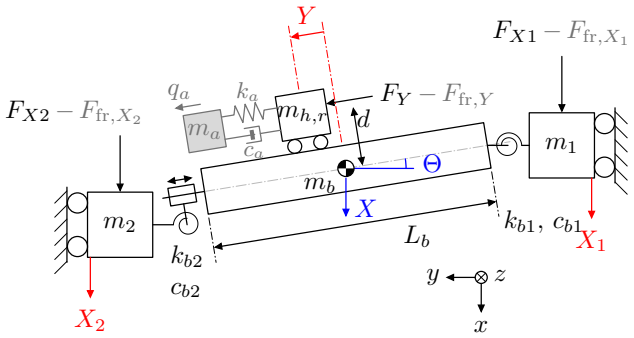



Fig. 6: Simulated system: the model of Fig. 1 with an added mass, spring and damper on the payload, and friction $F_{\mathrm{fr}}$ at the actuators. Grey elements represent the unmodelled dynamics of the baseline.

unmodelled dynamics to the system, for the augmented model to capture. For the second addition, we split the payload mass $m_h$ into $m_a$ and $m_{h,r} = m_h - m_a$, with $m_a$ connected to $m_{h,r}$ by a spring and damper in the $Y$ direction. This adds a vibration mode which the baseline model is not able to represent with its states. With the friction acting against each actuator force on the input, (4) becomes

$$\begin{aligned} M(Y)\ddot{q} + C\dot{q} + Kq &= P\big(u_{\mathrm{act}} - F_{\mathrm{fr}}(P^\top\dot{q})\big), \\ F_{\mathrm{fr},i}(v_i) &= c_{c,i}\tanh\big(v_i/v_{\mathrm{fr}}\big), \end{aligned} \tag{37a}$$

where $v_i$ is the velocity of actuator $i \in \{X_1, X_2, Y\}$, with the actuator velocities given by $P^\top\dot{q}$, and $v_{\mathrm{fr}}$ sets the velocity scale of the smoothing. For $|v_i| \gg v_{\mathrm{fr}}$, the friction approaches the Coulomb law $F_{\mathrm{fr},i} = c_{c,i}\operatorname{sign}(v_i)$ of [6]. The friction levels $c_c$ are taken from García-Herreros et al. [6, Table I].
The added mass adds the displacement $q_a$ of $m_a$ relative to $m_{h,r}$ along $Y$. We use the Lagrange-Euler equations of Section III-A, adding the kinetic energy of $m_a$, the elastic energy of $k_a$ and the dissipation of $c_a$. As in the baseline, we leave out the Coriolis and centripetal terms. With $q_{\mathrm{e}} = \begin{bmatrix} q^\top & q_a \end{bmatrix}^\top$, the system is

$$M_{\mathrm{e}}(Y, q_a)\,\ddot{q}_{\mathrm{e}} + C_{\mathrm{e}}\dot{q}_{\mathrm{e}} + K_{\mathrm{e}}q_{\mathrm{e}} = \begin{bmatrix} P\big(u_{\mathrm{act}} - F_{\mathrm{fr}}(P^\top\dot{q})\big) \\ 0 \end{bmatrix}, \tag{37b}$$

where $M_{\mathrm{e}}$, $C_{\mathrm{e}}$ and $K_{\mathrm{e}}$ are given in Appendix C.1. We simulate the system with fixed-step RK4 at $f_{\mathrm{sim}}$= 20 kHz. All parameter values are listed in Table VI.

### B. Closed loop and excitation

All data is generated in closed loop with the controller $K_1$ of Appendix C.1, which has a bandwidth of $f_b$ = 100 Hz. $K_{\mathrm{fb}}$ in (2) is $K_1$ discretised at $T_s$ with the Tustin rule. The feedforward $u^{\mathrm{ff}}$ in (14) is only the injected excitation force.

The added mass, spring and damper create an anti-resonance at 150 Hz and a resonance at 212 Hz in theopen-loop transfer function from $F_Y$ to $Y$. We have chosen both frequencies to be above the crossover of the loop with $K_1$, so that the feedback does not suppress them in the data.

Each record is 12 s long and uses one of five motion types: standstill at a fixed $Y$, slow sweeps of $Y$, $X$-$Y$ Lissajous paths, point-to-point moves with S-curve profiles, and operational motion profiles. The motion profiles excite the low frequencies up to the loop crossover (Table IX), and barely excite the added mass. We therefore add a multisine force to each of the generalised channels $X$, $\Theta$ and $Y$, except in the records with an operational motion profile. The multisine band is excited from the highest crossover frequency of the loop with $K_1$ at 106 Hz up to 297 Hz (Fig. 10). Above the crossover, this band covers 90 % of the squared difference between the frequency responses of the system and the baseline. Each record has its own random multisine phases, chosen for a low crest factor.

Independent coloured noise is added to the position measurements of each axis, $y = q_{\text{act}} + v$. At standstill, the simulated servo error matches the noise level and spectral shape of the ASMPT gantry. The noise is not modelled to capture the machine's resonances. The servo-error signal-to-noise ratio (SNR)of the training records ranges from 58 to 74 dB on the $X$ actuators and 71 to 80 dB on $Y$, depending on the motion profileand the multisine.

The records are downsampled to $T_s = 0.25$ ms. The servo error is filtered by an eighth-order Butterworth low-pass at 1.6 kHz, we apply it forward and backward. The force is averaged over each sample. The simulated system and controller $K_1$ run at $f_{\text{sim}} = 20$ kHz, while Section II assumes the loop runs at $T_s$. We evaluate the effect of this difference in Table IX.

### C. Records

We use 18 records for training, 6 for validation and 11 for testing (Table VIII in Appendix C.2). Training and validation include all five motion types. Their records stay within $|Y| \leq Y_{\text{tr}}$, 75 % of the acceleration limit and 75 % of the velocity limit. The validation records are only used to select checkpoints and hyperparameters.

We separate the test records into interpolation and extrapolation. Interpolation stays within the training range, while each extrapolation record exceeds at least one of the limits on $Y$, acceleration or velocity. E5 exceeds multiple limits. Each extrapolation record has the same motion as one interpolation record, to show the effect of leaving the training range. We also simulate their motion with a multisine, because the motion profiles barely excite the added mass. The results for all three sets are reported, but performance is evaluated on the test set.
.

### D. Models

The models of Table I are compared to answer each aspect of Section I. We identify $\mathcal{M}_{\text{U}}$ and $\mathcal{M}_{\text{OBC}}$ by minimising the loss of Section IV-C with the same training setup (Table XI). This includes PS to initialise the added states (Section IV-E). The parameter combinations $\theta_{\text{base}}$ of the detuned start are 10 % larger or smaller in magnitude than their true values, and every run uses the same signs. Every model is trained once, with the same seed.

TABLE I: Models.

| Model | Baseline | Learned states | OBC |
|---|---|---|---|
| $\mathcal{M}_{\text{LPV}}$ | true, not trained | 0 | no |
| $\mathcal{M}_{\text{U}}$ | trained | 2 | no |
| $\mathcal{M}_{\text{OBC}}$ | trained | 2 | yes |
| $\mathcal{M}_{\text{BB}}$ | none | 16 | no |

We compare the simulation error of $\mathcal{M}_{\text{OBC}}$ and $\mathcal{M}_{\text{LPV}}$, to test whether the augmentation is able to improve the performance by capturing the missing dynamics. To determine the number of added states and whether to use PS initialisation, we compare the simulation error of $\mathcal{M}_{\text{U}}$ for each combination. These runs start from the true parameters, so that parameter errors do not affect the choice.

The simulation error of $\mathcal{M}_{\text{OBC}}$ and $\mathcal{M}_{\text{U}}$ determines whether the constraint on the neural network limits the performance of the model, and whether OBC keeps the parameters interpretable. Both are trained from the true parameters and from the 10 % detuned start, to test whether the estimate depends on the start. We test generalisation by comparing the simulation error of $\mathcal{M}_{\text{OBC}}$, $\mathcal{M}_{\text{LPV}}$ and $\mathcal{M}_{\text{BB}}$ on the interpolation and extrapolation records.

The black box $\mathcal{M}_{\text{BB}}$ is a SUBNET model [15] that predicts the closed-loop servo error from the reference and injected force, without modelling the controller separately (Appendix C.4). It uses the same records, training window and encoder window as the grey-box models.

### E. Metrics

All models are simulated over the full records. The grey-box models use the closed loop of Section IV-B, while the black-box model uses the reference and injected force as inputs. We compare the predicted servo error with the recorded servo error $y - r$, using the known reference. The main metric is the NRMS of [15, Eq. (43)], adapted by using the servo error as output. The data is also generated without measurement noise, with output $y^\circ$. The NRMS is computed as

$$\begin{aligned} \text{RMS}_j &= \Big(\frac{1}{N-n-1}\sum_{k=n+1}^{N-1}(\hat{y}_{j,k} - y_{j,k})^2\Big)^{1/2}, \\ \text{NRMS}_{e,j} &= \frac{\text{RMS}_j}{\sigma_{e,j}}, \end{aligned} \tag{38}$$

where $j$ is the stage axis, $N$ the record length, $n+1$ the encoder window length and $\sigma_{e,j}$ the standard deviation of the servo error $y_j - r_j$for $k = n+1$ to $N-1$. An NRMS of 100 % means the model performs no better than predicting the mean servo error. The noise floor is taken as the $\text{NRMS}_e$ of (38), with $\hat{y}$ replaced by the noise-free output $y^\circ$.

To compare the parameters of $\mathcal{M}_{\text{U}}$ and $\mathcal{M}_{\text{OBC}}$, the relative error of the estimated combinations is computed against their true values, which are known in simulation. The overlap $\rho$ of (36) shows whether the network stays out of the parameter directions. It is computed on the training, validation and interpolation test records.

TABLE II: $\mathrm{NRMS}_e$ per axis on the test records, mean over the 10 records with a multisine and the 8 without. $\mathcal{M}_{\mathrm{OBC}}$ starts from the detuned parameters, the other grey-box models from the true parameters.

| | Multisine | | | No multisine | | |
|---|---|---|---|---|---|---|
| $\mathrm{NRMS}_e$ (%) | $X_1$ | $X_2$ | $Y$ | $X_1$ | $X_2$ | $Y$ |
| $\mathcal{M}_{\mathrm{LPV}}$ | 13.3 | 13.3 | 83.7 | 4.3 | 4.3 | 3.3 |
| $n_{\bar{x}} = 0$ | 9.5 | 8.5 | 60.4 | 5.0 | 4.3 | 19.9 |
| $n_{\bar{x}} = 2$, plain | 8.0 | 7.3 | 53.6 | 4.3 | 4.2 | 13.4 |
| $n_{\bar{x}} = 2$, PS | 7.1 | 6.8 | 6.1 | 4.0 | 3.9 | 4.0 |
| $n_{\bar{x}} = 8$, PS | 6.7 | 6.1 | 6.6 | 3.9 | 3.9 | 4.0 |
| $\mathcal{M}_{\mathrm{OBC}}$ | 7.7 | 7.1 | 6.2 | 4.5 | 4.5 | 3.7 |
| Noise floor | 0.05 | 0.04 | 0.01 | 0.04 | 0.04 | 0.01 |

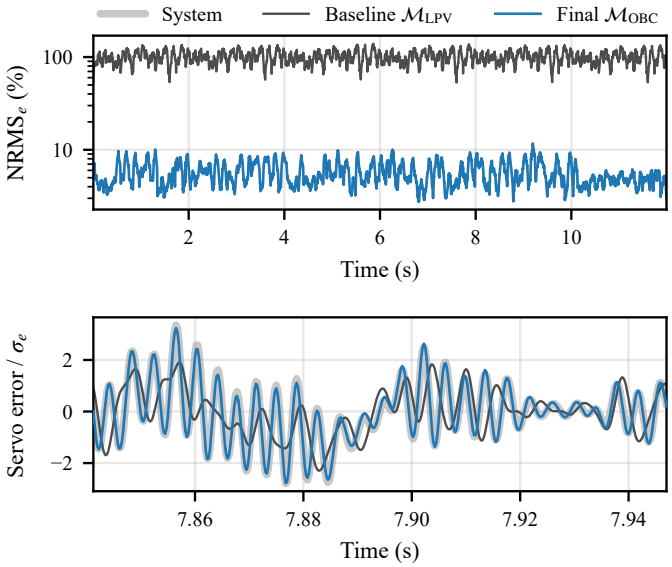


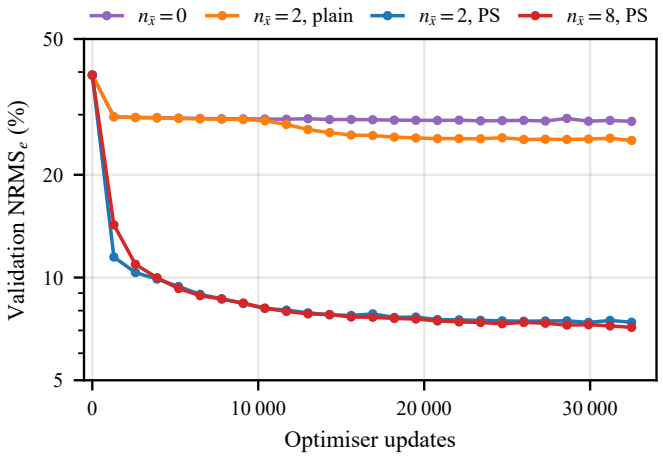

Fig. 7: Test record I1, $Y$ axis. Top: $\mathrm{NRMS}_e$ of (38) over 50 ms. Bottom: servo error $y-r$ of the system and $\hat{y}-r$ of the models, divided by $\sigma_{e,Y}$.

## VII. Results and Discussion

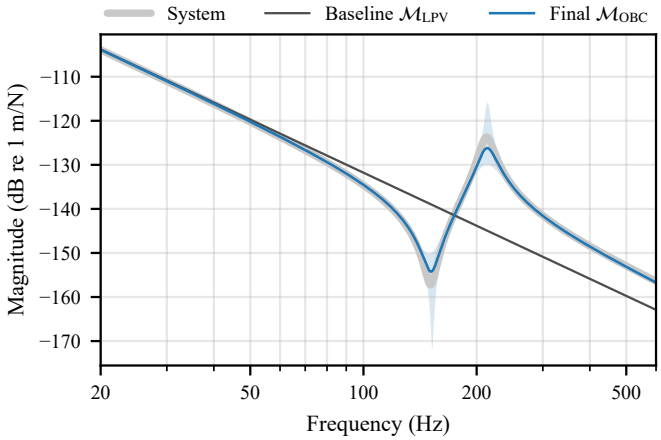
In this section, we analyse the models of Table I for each aspect of Section I. All errors are reported on the test set in $\mathrm{NRMS}_e$ of (38), against the noise floor. Appendix B gives the training and validation records.

### A. Augmentation structure

For Aspect 1, we compare the augmentation structures with the baseline. Table II compares the final grey box model $\mathcal{M}_{\mathrm{OBC}}$ with the baseline at the true parameters. For the data with multisine, the final model lowers the error on $Y$ by more than an order of magnitude and almost halves the error on $X_1$ and $X_2$. For the data without multisine both models achieve similar results. The final model has a lower error than the baseline on record I1 (Fig. 7). The final model is able to capture the resonance of the system (Fig. 8b).
Without added states or without PS, the error on $Y$ with multisine stays above 50 %. Two added states with PS reach 6.1 % and eight added states gives no significant improvement. Fig. 8a shows that the models trained with PS have a significantly lower validation error, compared to the other models during training. The missing dynamics are therefore only captured with added states and PS.




(a) Validation NRMS$_e$ during training, mean over the validation records and axes. Update 0 equals the baseline.




(b) Open-loop Bode magnitude from $F_Y$ to $Y$ at $Y = 0$, from the models linearised at standstill. Shaded: final model over $|Y| \leq 1.3\,Y_{\mathrm{tr}}$.

Fig. 8: Training and frequency response of the augmentation structures.

### B. *Interpretability*

For Aspect 2, we compare the estimates of $\mathcal{M}_{\mathrm{OBC}}$ and $\mathcal{M}_{\mathrm{U}}$. Table III gives the estimates from the detuned and the true start. The estimates of $\mathcal{M}_{\mathrm{OBC}}$ from the two starts differ by 31.3 %, while those of $\mathcal{M}_{\mathrm{U}}$ differ by 36.4 %. The final model with the orthogonality constraint and the unconstrained model have similar performance. OBC keeps the network out of the parameter directionson the training records: its network overlap is much lower than that of $\mathcal{M}_{\mathrm{U}}$(Table IV). The network overlap of $\mathcal{M}_{\mathrm{OBC}}$ is also much lower than that of $\mathcal{M}_{\mathrm{U}}$ on the validation and test records. It increases from 0.00 on the training records to 0.24 on the test records. However, the missing dynamics are not orthogonal to these directions, their overlap is 0.59. Both models have deviated further from the true combinations than their initial values. With the detuned start, $\mathcal{M}_{\mathrm{U}}$ deviates 83.1 % from the true values and $\mathcal{M}_{\mathrm{OBC}}$ 69.1 %. The parameter combinations describing the damping deviate the most, while the mass combinations stay close to their true values. The estimates of both models still depend on where training starts.

TABLE III: Relative error of the ten estimated combinations of $\mathcal{M}_{\mathrm{U}}$ and $\mathcal{M}_{\mathrm{OBC}}$, trained from the detuned and from the true start. Distances: Euclidean norm over the ten combinations.

| | | $\mathcal{M}_{\mathrm{U}}$ | | $\mathcal{M}_{\mathrm{OBC}}$ | |
|---|---|---|---|---|---|
| Rel. error (%) | Detuned start | Detuned | True | Detuned | True |
| $k_{b,\Sigma}$ | −10.0 | −21.8 | −7.6 | −7.1 | −9.7 |
| $c_{g1}$ | +10.0 | +45.6 | +27.7 | +34.0 | +23.6 |
| $c_{g2}$ | +10.0 | +45.9 | +27.2 | +28.4 | +16.2 |
| $c_y$ | +10.0 | +43.3 | +29.5 | +47.4 | +34.1 |
| $c_{b,\Sigma}$ | −10.0 | +14.7 | +29.2 | +21.8 | +35.7 |
| $m_h$ | −10.0 | −1.3 | +1.4 | −2.0 | +1.8 |
| $m_\Sigma$ | +10.0 | +2.6 | +1.3 | +0.5 | −0.6 |
| $m_\Delta$* | −0.5 | −0.5 | −0.1 | −2.1 | −1.6 |
| $J_{\mathrm{eff}}$ | −10.0 | −2.2 | +4.1 | −2.7 | +4.7 |
| $d$ | −10.0 | −11.7 | −9.1 | +3.7 | −12.7 |
| Distance to truth | 30.0 | 83.1 | 58.3 | 69.1 | 59.5 |
| Distance between starts | | 36.4 | | 31.3 | |

* relative to the mean actuator mass, because its true value is close to zero.

TABLE IV: Overlaps of (36) and $\mathrm{NRMS}_e$ of $\mathcal{M}_{\mathrm{U}}$ and $\mathcal{M}_{\mathrm{OBC}}$, trained from the detuned start. The $\mathrm{NRMS}_e$ is the mean over the records with and without multisine.

| | $\mathcal{M}_{\mathrm{U}}$ | $\mathcal{M}_{\mathrm{OBC}}$ |
|---|---|---|
| $\rho(F_{\mathrm{aug}})$, training records | 0.85 | 0.00 |
| $\rho(F_{\mathrm{aug}})$, validation records | 0.84 | 0.08 |
| $\rho(F_{\mathrm{aug}})$, interpolation test records | 0.84 | 0.24 |
| $\rho(\Xi)$, training records | 0.51 | |
| $\rho(\Xi)$, validation records | 0.57 | |
| $\rho(\Xi)$, interpolation test records | 0.59 | |
| $\mathrm{NRMS}_e$ (%), 10 records with a multisine | | |
| $X_1$ | 7.1 | 7.7 |
| $X_2$ | 6.5 | 7.1 |
| $Y$ | 6.4 | 6.2 |
| $\mathrm{NRMS}_e$ (%), 8 records without | | |
| $X_1$ | 4.0 | 4.5 |
| $X_2$ | 3.9 | 4.5 |
| $Y$ | 4.0 | 3.7 |

### C. Generalisation

For Aspect 3, we compare the final model with the baseline and the black box on the interpolation (I) and extrapolation (E) records (Table V). The errors between E and I differ only by at most 0.5 points. Without multisine the model performs as well as or slightly worse than the baseline. The black box is worse than the final model with and without multisine, and reaches 97 to 128 % without it. It even performs worse than the baseline model, except on $Y$ with multisine. With multisine, the final model stays below 8 % on all axes, while the errors of the baseline and the black box exceed 65 % on $Y$. The final model generalises to the held-out positions and unseen motion profiles, but only improves on the baseline when the multisine excites the missing dynamics.

TABLE V: $\mathrm{NRMS}_e$ per axis on the test records, mean over the interpolation (I) and extrapolation (E) records with and without a multisine.

| | | Multisine | | No multisine | |
|---|---|---|---|---|---|
| $\mathrm{NRMS}_e$ (%) | | I | E | I | E |
| $\mathcal{M}_{\mathrm{LPV}}$ | $X_1$ | 14.3 | 12.3 | 4.5 | 4.1 |
| | $X_2$ | 14.6 | 12.0 | 4.6 | 4.1 |
| | $Y$ | 89.5 | 78.0 | 3.4 | 3.1 |
| $\mathcal{M}_{\mathrm{OBC}}$ | $X_1$ | 7.9 | 7.5 | 4.7 | 4.3 |
| | $X_2$ | 7.3 | 6.8 | 4.7 | 4.3 |
| | $Y$ | 6.1 | 6.3 | 3.8 | 3.5 |
| $\mathcal{M}_{\mathrm{BB}}$ | $X_1$ | 78.5 | 82.4 | 104.8 | 108.5 |
| | $X_2$ | 79.2 | 81.1 | 97.0 | 99.8 |
| | $Y$ | 65.9 | 74.7 | 127.7 | 127.0 |

### D. Discussion

The improvement on $Y$ with multisine is mainly caused by the learned vibration mode of the added mass(Fig. 8b). The static correction without added states, is only able to lower the error on $Y$ from 83.7 % for the baseline to 60.4 %, while two added states with PS reach 6.1 % (Table II). Training without PS does not seem to find this mode in the current setting, it stays above 50 %. Without multisine, the final model performs slightly worse than the baseline. For these records the baseline error is only a few percent of the serve error, and only four of the eighteen training records are without multisine. Resulting in the records without multisine contributing relatively little to the loss. The final model remains far above the noise floor on every record. A possible reason is that the relative error is already small, so reducing it further requires a better tuned optimisation. The augmentation might not be able to capture the unmodelled friction, or the difference in controller rates contributes to this. From these results, we cannot conclude what the reason is.

OBC is able to keep the neural network out of the parameter directions on the training tuples, and for the most part on the validation and test records compared to $\mathcal{M}_{\mathrm{U}}$ (Table IV). However, the missing dynamics are not orthogonal to these directions, their overlap is 0.59 (Table IV). Orthogonality is defined to hold only to first order and at $\bar{x} = 0$, and the added states enter through $f_{\mathrm{aug}}$ without projection. This holds for a single step, not for the simulated output, where the model is simulated over multiple steps with nonzero added states. Even when training starts from the true parameters, the damper combinations deviate the most, while the mass combinations stay close to their true values (Table III). A possible explanation is that the dampers absorb part of the friction, since both produce a force in the opposite direction of the velocity. For both models, the estimates from the two starts differ by a similar amount (Table III), so the trained estimate still depends on the start. Given the current results, the estimates cannot be used to trace a change in the machine back to a physical parameter.

For the interpolation and extrapolation records both the baseline and the final model have a similar error (Table V), However,

most extrapolation records extend beyond the training range in only one direction. The black box model underperforms, while Hoekstra et al. report similar results for black-box and dynamic augmented models [5, Table 8]. Possible reasons for the underperformance are that it starts from a random initialisation, instead of for example a best linear approximation. The black box model also has to learn the controller dynamics, because the controller is not included in the training loop. What we do see is that the black box model performs much better on the records with multisine than on the records without (Table VII). For the records without multisine, the reference is the only input. The black box model therefore has to learn the complete closed-loop response, while the grey box model uses the known controller and starts from the baseline.

The results are limited to simulation with one seed and the same controller used to generate the data. Training and evaluation are both performed in closed loop, where the controller can suppress errors in the plant model. A different controller might be able to reveal these errors. The recorded input is correlated with the measurement noise through the feedback, while SUBNET assumes they are independent [15]. Therefore, we have not shown the consistency of the estimate.

For future work, the model should be evaluated on measured data, and evaluated with a different controller. OBC could also be extended to the simulated output, using its Jacobian over a training window instead of the Jacobian of a single state update.

## VIII. Conclusion

For the dual-drive gantry, a parallel augmentation with added states is able to capture the vibration mode of the added mass, spring and damper when it is trained in closed loop with the known controller and the added states are initialised with PS. The resulting model reduces the prediction error on $Y$ when this mode is sufficiently excited in the data. The final model performs slightly worse than the baseline model on records without multisine excitation. The model generalises to held-out positions and unseen motion profiles: the prediction errors on the interpolation and extrapolation records are similar, and the final model performs better than the black box model. Compared to unconstrained estimation, OBC is able to reduce the overlap between the neural network and the parameter directions. However, the parameter estimates still deviate from their true values and depend on where training starts. In conclusion, a parallel augmentation with added states initialised with PS improves the prediction of the gantry, but the uniqueness provided by OBC for a single state update does not make the trained parameters interpretable as defined in Section I. These results are limited to simulation with one controller.

## References


[1] J. Hwang and S. D. Noh, "Digital twin-based optimization of operational parameters for cluster tools in semiconductor manufacturing," *IEEE Access*, vol. 12, pp. 122 078–122 100, 2024.

[2] International Roadmap for Devices and Systems (IRDS), "International roadmap for devices and systems: 2022 edition executive summary," IEEE, Tech. Rep., 2022.

[3] T. Workman, L. Mirkarimi, J. Theil, G. Fountain, K. M. Bang, B. Lee, C. Uzoh, D. Suwito, G. Gao, and P. Mrozek, "Die to wafer hybrid bonding and fine pitch considerations," in *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 2021, pp. 2071–2077.

[4] B. M. Kessels, "Ai-based model updating for nonlinear dynamical systems," PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.

[5] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Learning-based augmentation of first-principle models: A linear fractional representation-based approach," 2026, submitted to Automatica.

[6] I. García-Herreros, X. Kestelyn, J. Gomand, R. Coleman, and P.-J. Barre, "Model-based decoupling control method for dual-drive gantry stages: A case study with experimental validations," *Control Engineering Practice*, vol. 21, pp. 298–307, 2013.

[7] C. S. Teo, K. K. Tan, S. Y. Lim, S. Huang, and E. B. Tay, "Dynamic modeling and adaptive control of a h-type gantry stage," *Mechatronics*, vol. 17, pp. 361–367, 2007.

[8] B. M. Györök, M. Schoukens, T. Péni, and R. Tóth, "Orthogonal-by-construction augmentation of physics-based input-output models," *IFAC Journal of Systems and Control*, vol. 35, p. 100376, 2026.

[9] R. Isermann, "Fault diagnosis of machines via parameter estimation and knowledge processing—tutorial paper," *Automatica*, vol. 29, no. 4, pp. 815–835, 1993.

[10] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, Nov. 2022.

[11] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[13] J. P. Noël and J. Schoukens, "Grey-box state-space identification of nonlinear mechanical vibrations," *International Journal of Control*, vol. 91, no. 5, pp. 1118–1139, 2018.

[14] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, "Learning-based augmentation of physics-based models: an industrial robot use case," *Data-Centric Engineering*, vol. 5, p. e12, 2024.

[15] G. I. Beintema, M. Schoukens, and R. Tóth, "Deep subspace encoders for nonlinear system identification," *Automatica*, vol. 156, p. 111210, 2023.

[16] R. Tóth, *Modeling and Identification of Linear Parameter-Varying Systems*, ser. Lecture Notes in Control and Information Sciences. Berlin, Heidelberg: Springer, 2010, vol. 403.

[17] R. Drenth, J. H. Hoekstra, M. Schoukens, and R. Tóth, "Efficient learning of affine and rational dependency LPV models with linear fractional representation," 2026.

[18] M. Gautier and W. Khalil, "Direct calculation of minimum set of inertial parameters of serial robots," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 3, pp. 368–373, 1990.

[19] C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford: Clarendon Press, 1995.

[20] U. Forssell and L. Ljung, "Closed-loop identification revisited," *Automatica*, vol. 35, no. 7, pp. 1215–1241, 1999.

[21] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Encoder initialisation methods in the model augmentation setting," 2026.

[22] A. Hefny, C. Downey, and G. J. Gordon, "Supervised learning for dynamical system learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[23] C. Downey, A. Hefny, B. Li, B. Boots, and G. Gordon, "Predictive state recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[24] A. Venkatraman, N. Rhinehart, W. Sun, L. Pinto, M. Hebert, B. Boots, K. M. Kitani, and J. A. Bagnell, "Predictive-state decoders: Encoding the future into recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[25] B. M. Györök, J. H. Hoekstra, J. Kon, T. Péni, M. Schoukens, and R. Tóth, "Orthogonal projection-based regularization for efficient model augmentation," in *Proceedings of the 7th Annual Conference on Learning for Dynamics and Control*, ser. Proceedings of Machine Learning

Research, N. Ozay, L. Balzano, D. Panagou, and A. Abate, Eds., vol. 283, 2025.

## APPENDIX A
## SUPPORTING DERIVATIONS

### A.1 Nominal physical parameters

Table VI lists the nominal values used by the baseline. These values are from the dual-drive gantry of García-Herreros et al. [6, Table I]. The lower block shows the parameters of the simulated system, which are omitted from the baseline (Section VI-A).

TABLE VI: Baseline parameters (upper block) and the additional Coulomb friction and absorber parameters of the simulated system (lower block).

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| $m_1$ | 10.2 kg | $m_2$ | 10.7 kg |
| $m_b$ | 22.8 kg | $m_h$ | 10.1 kg |
| $J_b$ | 1.00 kg m$^2$ | $J_h$ | 0.05 kg m$^2$ |
| $c_{g1}$ | 14.5 N s/m | $c_{g2}$ | 20.3 N s/m |
| $c_y$ | 10.0 N s/m | $d$ | 0.10 m |
| $c_{b1}$ | 9.0 N m s/rad | $c_{b2}$ | 9.0 N m s/rad |
| $k_{b1}$ | 1987.5 N m/rad | $k_{b2}$ | 1987.5 N m/rad |
| $L_b$ | 0.725 m | | |
| $c_{c,X_1}$ | 16.80 N | $c_{c,X_2}$ | 18.35 N |
| $c_{c,Y}$ | 11.60 N | $v_{\text{fr}}$ | 1 mm/s |
| $m_a$ | 0.5 $m_h$ | $L_0$ | 0.10 m |
| $f_a$ | 150 Hz | $\zeta_a$ | 0.03 |

### A.2 LPV-LFR realisation of the baseline

This appendix derives an exact LPV-LFR realisation of the baseline (7) and shows that it is well posed whenever $M(Y)$ is nonsingular. In (6), $M_0$ is the part of (5a) without $Y$, and

$$M_1 = \begin{bmatrix} 0 & -m_h & 0 \\ -m_h & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & m_h & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

An LFR is introduced to separate the rational scheduling dependence of (7), with a linear time-invariant (LTI) part whose matrices do not depend on $Y$. We move the dependence out of the LTI part $G$ and into a Delta-block $\Delta(Y)$, with the scheduling being connected in feedback with $G$ (Fig. 9) [5], [17]. Substituting (6) into (4) gives

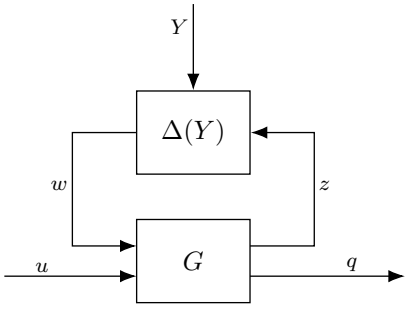



Fig. 9: Self-scheduled baseline interconnection. The Delta-block closes the latent variables of $G$ through $w = \Delta(Y)z$, with $\Delta(Y) = YI_6$ and $Y$ a component of the state.

$$M_0\ddot{q} + YM_1\ddot{q} + Y^2M_2\ddot{q} = u - C\dot{q} - Kq. \tag{39}$$

Here $Y$ only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable $Y^2$,

$$\ddot{q} \xrightarrow{Y} a_1 \xrightarrow{Y} a_2, \tag{40}$$

with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$. We treat $a_1$ and $a_2$ as independent signals, so (39) becomes

$$M_0\ddot{q} + M_1a_1 + M_2a_2 = u - C\dot{q} - Kq, \tag{41}$$

which contains no $Y$. This is allowed, because the Delta-block below imposes $a_1 = Y\ddot{q}$ and $a_2 = Ya_1$ again. The latent variables are $z = [\ddot{q}^\top \ a_1^\top]^\top$ and $w = [a_1^\top \ a_2^\top]^\top$, with the Delta-block mapping $z$ to $w$,

$$w = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} Y\ddot{q} \\ Ya_1 \end{bmatrix} = \underbrace{\begin{bmatrix} YI_3 & 0 \\ 0 & YI_3 \end{bmatrix}}_{\Delta(Y)=YI_6} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \Delta(Y)z. \tag{42}$$

We write the LTI part $G$ of the LFR in continuous time, adapting the discrete-time block structure of [17, Eq. (2)]:

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = G \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}, \tag{43}$$

where $\tilde{x} = [q^\top \ \dot{q}^\top]^\top$ is the state and $q$ the output. The first block row is the state equation, with $\ddot{q}$ obtained from (41) by multiplying with $M_0^{-1}$. Conversely, closing the algebraic loop with (42) results in $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (41) back into (4). The LFR is therefore an exact realisation of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every $Y$ [17, Def. 1].

With $R = M_0^{-1}$ ($M_0 = M(0) \succ 0$, Appendix A.3), (41) gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1a_1 - M_2a_2)$. Partitioning $G$ in (43) as in [17, Eq. (2)], with rows $(\dot{\tilde{x}}, z, q)$ and columns $(\tilde{x}, w, u)$, gives the blocks

$$\begin{aligned} A_x &= \begin{bmatrix} 0 & I_3 \\ -RK & -RC \end{bmatrix}, & B_w &= \begin{bmatrix} 0 & 0 \\ -RM_1 & -RM_2 \end{bmatrix}, & B_u &= \begin{bmatrix} 0 \\ R \end{bmatrix}, \\ C_z &= \begin{bmatrix} -RK & -RC \\ 0 & 0 \end{bmatrix}, & D_{zw} &= \begin{bmatrix} -RM_1 & -RM_2 \\ I_3 & 0 \end{bmatrix}, & D_{zu} &= \begin{bmatrix} R \\ 0 \end{bmatrix}, \\ C_y &= \begin{bmatrix} I_3 & 0 \end{bmatrix}, & D_{yw} &= 0, & D_{yu} &= 0. \end{aligned} \tag{44}$$

Here $\tilde{x}, z, w \in \mathbb{R}^6$ and $u, q \in \mathbb{R}^3$, so $G \in \mathbb{R}^{15\times15}$.

Substituting $w = \Delta(Y)z$ gives the loop equation $(I_6 - D_{zw}\Delta(Y))z = C_z\tilde{x} + D_{zu}u$, with

$$I_6 - D_{zw}\Delta(Y) = \begin{bmatrix} I_3 + YRM_1 & YRM_2 \\ -YI_3 & I_3 \end{bmatrix}.$$

The Schur complement of the lower identity block gives

$$\begin{aligned} \det\big(I_6 - D_{zw}\Delta(Y)\big) &= \det\big(I_3 + YRM_1 + Y^2RM_2\big) \\ &= \det M(Y)/\det M_0, \end{aligned}$$

so the condition of [17, Def. 1] holds if and only if $M(Y)$ is nonsingular.

### A.3 Positive definiteness of the inertia matrix

With $a = L_b m_\Delta/2$, (5a) in terms of (8) is

$$M(Y) = \begin{bmatrix} m_\Sigma + m_h & a - m_h Y & 0 \\ a - m_h Y & J_{\text{eff}} + m_h(d^2 + Y^2) & -m_h d \\ 0 & -m_h d & m_h \end{bmatrix}.$$

By Sylvester's criterion, applied to the principal minors from the lower-right corner, $M(Y) \succ 0$ if and only if

$$m_h > 0, \qquad m_h\big(J_{\text{eff}} + m_h Y^2\big) > 0, \qquad \det M(Y) > 0,$$

with

$$\begin{aligned}\frac{\det M(Y)}{m_h} &= (m_\Sigma + m_h)(J_{\text{eff}} + m_h Y^2) - (a - m_h Y)^2 \\ &= m_\Sigma m_h\Big(Y + \frac{a}{m_\Sigma}\Big)^2 + (m_\Sigma + m_h)\Big(J_{\text{eff}} - \frac{a^2}{m_\Sigma}\Big).\end{aligned}$$

For $m_h, m_\Sigma, J_{\text{eff}} > 0$, this expression is positive for every $Y$ if and only if (9) holds. Substituting the definitions of $m_\Sigma$, $m_\Delta$ and $J_{\text{eff}}$ gives

$$\begin{aligned}m_\Sigma J_{\text{eff}} - \tfrac{L_b^2}{4} m_\Delta^2 &= m_\Sigma(J_b + J_h) \\ &\quad + \tfrac{L_b^2}{4}\big[4m_1 m_2 + m_b(m_1 + m_2)\big] > 0.\end{aligned}$$

In (10), with $b = \gamma(2/L_b)\sqrt{m_\Sigma J_{\text{eff}}}$ and $b_0$ its value at $\theta^0_{\text{base}}$,

$$r_0 = \frac{m_{\Delta,0}}{b_0}, \quad |r_0| < 1, \quad \eta_0 = \operatorname{atanh}(r_0), \quad s = \frac{m_{\Delta,0}}{b_0(1 - r_0^2)},$$

so that at $\beta = 0$, $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta/\partial \beta_{m_\Delta} = m_{\Delta,0}$.

### A.4 Identifiable combinations

The matrices $M_0$, $M_1$, $M_2$, $C$ and $K$ depend on the combinations in $\theta_{\text{base}}$ as follows:

$$\begin{aligned} M_{0,11} &= m_\Sigma + m_h, & M_{0,12} &= \tfrac{L_b}{2} m_\Delta, \\ M_{0,22} &= J_{\text{eff}} + m_h d^2, & M_{0,23} &= -m_h d, \\ C_{11} &= c_{g1} + c_{g2}, & C_{33} &= c_y, \\ C_{12} &= \tfrac{L_b}{2}(c_{g1} - c_{g2}), & K_{22} &= k_{b,\Sigma}, \\ M_{0,33} &= -M_{1,12} = M_{2,22} = m_h, \\ C_{22} &= c_{b,\Sigma} + \tfrac{L_b^2}{4}(c_{g1} + c_{g2}). \end{aligned}$$

where $M_{i,jk}$ denotes entry $(j,k)$ of $M_i$. For $m_h > 0$, we can recover these combinations from the matrix entries:

$$\begin{aligned} m_h &= M_{0,33}, & d &= -M_{0,23}/m_h, \\ m_\Sigma &= M_{0,11} - m_h, & m_\Delta &= 2M_{0,12}/L_b, \\ J_{\text{eff}} &= M_{0,22} - m_h d^2, & k_{b,\Sigma} &= K_{22}, \\ c_{g1} &= \tfrac{1}{2}C_{11} + C_{12}/L_b, & c_{g2} &= \tfrac{1}{2}C_{11} - C_{12}/L_b, \\ c_{b,\Sigma} &= C_{22} - \tfrac{L_b^2}{4}C_{11}, & c_y &= C_{33}. \end{aligned}$$

Therefore, two different values of $\theta_{\text{base}}$ cannot give the same matrices.

## Appendix B
## Training, validation and test results

Table VII gives the models of Section VII on the training, validation and test records.

TABLE VII: $\text{NRMS}_e$ per axis on the training, validation and test records, mean over the records of each set with and without multisine.

| | Multisine | | | No multisine | | |
|---|---|---|---|---|---|---|
| $\text{NRMS}_e$ (%) | $X_1$ | $X_2$ | $Y$ | $X_1$ | $X_2$ | $Y$ |
| *Training records* (14 with, 4 without multisine) | | | | | | |
| $\mathcal{M}_{\text{LPV}}$ | 21.9 | 24.5 | 120.0 | 4.6 | 5.0 | 3.7 |
| $\mathcal{M}_{\text{U}}$ | 9.4 | 10.2 | 6.6 | 4.1 | 4.3 | 4.3 |
| $\mathcal{M}_{\text{OBC}}$ | 10.2 | 10.8 | 6.3 | 4.5 | 4.8 | 4.2 |
| $\mathcal{M}_{\text{BB}}$ | 32.4 | 34.5 | 25.2 | 75.6 | 76.9 | 104.4 |
| *Validation records* (4 with, 2 without multisine) | | | | | | |
| $\mathcal{M}_{\text{LPV}}$ | 23.0 | 24.9 | 121.8 | 4.7 | 4.9 | 3.7 |
| $\mathcal{M}_{\text{U}}$ | 9.9 | 10.2 | 6.6 | 4.1 | 4.2 | 4.3 |
| $\mathcal{M}_{\text{OBC}}$ | 10.8 | 10.8 | 6.5 | 4.5 | 4.8 | 4.2 |
| $\mathcal{M}_{\text{BB}}$ | 31.6 | 33.6 | 23.8 | 75.6 | 76.9 | 103.3 |
| *Test records* (10 with, 8 without multisine) | | | | | | |
| $\mathcal{M}_{\text{LPV}}$ | 13.3 | 13.3 | 83.7 | 4.3 | 4.3 | 3.3 |
| $\mathcal{M}_{\text{U}}$ | 7.1 | 6.5 | 6.4 | 4.0 | 3.9 | 4.0 |
| $\mathcal{M}_{\text{OBC}}$ | 7.7 | 7.1 | 6.2 | 4.5 | 4.5 | 3.7 |
| $\mathcal{M}_{\text{BB}}$ | 80.5 | 80.2 | 70.3 | 106.7 | 98.4 | 127.4 |

## Appendix C
## Experiment design details

### C.1 Data-generating system

The matrices of (37b) are

$$M_{\text{e}}(Y, q_a) = \begin{bmatrix} M(Y) + M_a(Y,\ell) & m_a\begin{bmatrix}0 & -d & 1\end{bmatrix}^\top \\ m_a\begin{bmatrix}0 & -d & 1\end{bmatrix} & m_a \end{bmatrix},$$

$$M_a(Y,\ell) = m_a \ell \begin{bmatrix} 0 & -1 & 0 \\ -1 & 2Y + \ell & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

$$C_{\text{e}} = \operatorname{diag}(C, c_a), \quad K_{\text{e}} = \operatorname{diag}(K, k_a),$$

where $M(Y)$ is (5a) with $m_h$ the total payload mass, $\ell = L_0 + q_a$ and $L_0$ is the offset of the added mass along $Y$. $k_a = m_a(2\pi f_a)^2$ and $c_a = 2\zeta_a\sqrt{k_a m_a}$, with natural frequency $f_a$ and damping ratio $\zeta_a$.

The controller of every record is, per stage axis $j$,

$$K_{1,j}(s) = \kappa_j \, \frac{s + \omega_b/6}{s} \, \frac{s + \omega_b/3}{s + 3\omega_b} \, \frac{10\,\omega_b}{s + 10\,\omega_b}, \tag{45}$$

where $\omega_b = 2\pi f_b$ is the bandwidth and $\kappa_j$ sets the open-loop gain of axis $j$ to one at $\omega_b$. $K_1$ is discretised by the Tustin rule at $f_{\text{sim}}$for the data, and by the same rule at $T_s$ for $K_{\text{fb}}$ in the model.

The signal-to-noise ratio of Section VI-A is $\text{SNR}_j = 20\log_{10}\big(\text{RMS}(y_j^\circ - r_j)/\text{RMS}(y_j - y_j^\circ)\big)$ over the training records.

### C.2 Excitation and records

Table VIII lists the records.

### C.3 Processing and checks

Table IX gives the checks of the design.

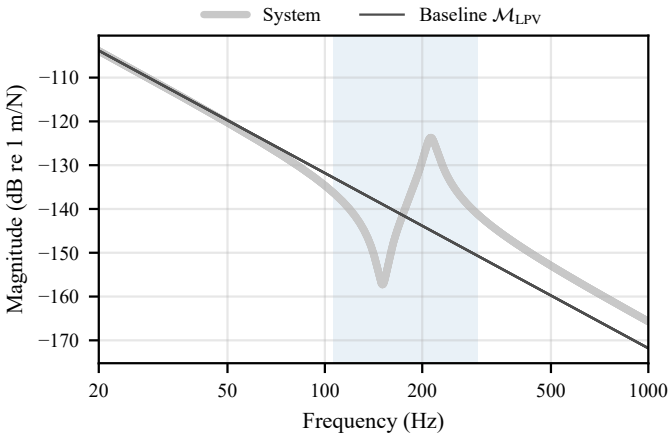



Fig. 10: Open-loop Bode magnitude from $F_Y$ to $Y$ of the system and the baseline at the five training positions. Shaded: multisine band.

TABLE VIII: Training, validation and test records.

| Record | Motion | Multisine | New | Pair |
|---|---|---|---|---|
| Training | all five types | 14 of 18 | | |
| Validation | all five types | 4 of 6 | positions, draws | |
| I1 | point-to-point | yes | phases, setpoints | |
| I2 | standstill | yes | position | |
| I3 | operational | no† | motion type | |
| I4 | route | no | motion type | |
| I5 | route | no† | motion type | |
| I6 | route | no† | motion type | |
| E1 | standstill | yes | $\lvert Y\rvert = 1.3\,Y_{\mathrm{tr}}$ | I2 |
| E2 | operational | no† | stroke of $Y$ | I3 |
| E3 | route | no† | acceleration, 100 % | I5 |
| E4 | route | no† | velocity, 98 % | I6 |
| E5 | operational | no† | several, 131 % acc. | I3 |

I: interpolation, inside the training range. E: extrapolation, outside it in the setting under New, with the motion of the Pair record. New: what differs from the training records, for E from the Pair record. E5 differs in more than one setting and is a stress record. †Also simulated with a multisine (I3M, E2M, and so on), at equal amplitude within a pair and 0.30 of the training amplitude for I6 and E4.

### C.4 Black-box model

The black box of Section VI-D is

$$x^{\mathrm{bb}}_{k+1} = f_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x^{\mathrm{bb}}_k, r_k, u^{\mathrm{ms}}_k\big), \quad \hat{y}_k = r_k + h_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x^{\mathrm{bb}}_k\big), \tag{46}$$

where $x^{\mathrm{bb}}_k \in \mathbb{R}^{n_x}$ and an encoder estimates $x^{\mathrm{bb}}$ at the window start from past inputs and servo errors, as in [15]. Its settings are the black-box rows of Table XI. Table X gives the configurations that were searched.

### C.5 Hyperparameters

TABLE IX: Checks of the experiment design.

| Check | Value |
|---|---|
| *Simulated loop, linearised at the training positions* | |
| Phase-margin loss, 4 / 2 kHz | 3.8° / 8.6° |
| *Training records* | |
| SNR 0.24 $f_b$ to 1.05 $f_b$, without multisine | ≥59 dB |
| $K_1$ at $T_s$, force error, median | 0.5 % |
| largest; without multisine | 1.0 %; 0.03 % |
| Force change within one hold of $T_s$, median | 10.4 % |

TABLE X: Black-box search: the 11 configurations, each trained for at most 80 000 updates, and their best validation $\mathrm{NRMS}_e$. Failed: the training loss became non-finite.

| # | $n_x$ | Width | LR | Param. | $\mathrm{NRMS}_e$ (%) | Best update | Status |
|---|---|---|---|---|---|---|---|
| 0 | 8 | 8 | $3\cdot10^{-3}$ | 7494 | 63.4 | 8000 | completed |
| 1 | 8 | 16 | $3\cdot10^{-3}$ | 13422 | 59.9 | 28000 | completed |
| 2 | 8 | 32 | $3\cdot10^{-3}$ | 27582 | 63.4 | 20000 | completed |
| 3 | 16 | 8 | $3\cdot10^{-3}$ | 10270 | 52.5 | 50000 | completed |
| 4 | 16 | 16 | $3\cdot10^{-3}$ | 16518 | 56.2 | 11000 | failed |
| 5 | 16 | 32 | $3\cdot10^{-3}$ | 31318 | 54.8 | 75000 | completed |
| 6 | 32 | 8 | $3\cdot10^{-3}$ | 16206 | 58.2 | 9000 | completed |
| 7 | 32 | 16 | $3\cdot10^{-3}$ | 23094 | 57.5 | 3000 | completed |
| 8 | 32 | 32 | $3\cdot10^{-3}$ | 39174 | 59.1 | 4000 | completed |
| **9** | **16** | **16** | $1\cdot10^{-3}$ | **16518** | 48.2 | **78000** | **completed** |
| 10 | 16 | 16 | $1\cdot10^{-2}$ | 16518 | 66.4 | 1000 | failed |

Bold: selected on validation (lowest $\mathrm{NRMS}_e$; within 2 % the fewer parameters).

TABLE XI: Hyperparameters of the identification.

| Setting | Value |
|---|---|
| *Model and data* | |
| Model rate $1/T_s$ | 4 kHz |
| RK4 steps per $T_s$ | 1 |
| Window $n_f$ | 400 (0.1 s) |
| Window stride | 10 samples |
| Encoder window $n+1$ | 30 samples |
| Added states $n_{\bar{x}}$ | 0, 2, 8 |
| Hidden layers, width $n_h$ (network, encoder correction) | 2, 16, tanh |
| Margin $\gamma$ | 0.99 |
| Detuning of the combinations | 10 % |
| *Optimiser* | |
| Adam rate | $10^{-5}$ |
| Batch | 512 windows |
| Updates | 33 400 |
| Validation interval | 1300 updates |
| L-BFGS iterations, history | 100, 50 |
| L-BFGS inner steps, windows | 20, 16384 |
| L-BFGS tolerance (grad., change) | $2.6 \cdot 10^{-5}$, $2.6 \cdot 10^{-7}$ |
| Seeds per model | 1 |
| *PS (Section IV-E)* | |
| S1 and S2 Adam rate | $10^{-3}$ |
| S1 batch | 64 windows |
| Decoder width | 16 |
| Residual horizon $H$ | $n+1$ |
| Left out records | last per class with at least 3 records |
| Check of the decoder | every 25 updates, 128 left-out windows |
| S1 length | 100 to 350 updates |
| S1 stops improving | less than 2 % over 4 checks |
| S2 runs if the decoder explains | at least 10 %, also checked at update 150 |
| S2 pairs $N_{\mathcal{P}}$ (fit, left out) | 8192, 2048 |
| S2 batch | 512 |
| S2 length | 500 to 8000 updates |
| S2 stops improving | less than 1 % over 5 checks of 100 updates |
| *Black box (Section VI-D)* | |
| States $n_x$ | 16 |
| Width of $f_{\mathrm{bb}}$, $h_{\mathrm{bb}}$ (encoder $2\times$) | 16 |
| Adam rate | $10^{-3}$ |
| Batch, stride | 256, 25 |
| Updates | at most 80 000 |
| Selection | lowest validation $\mathrm{NRMS}_e$ (Table X) |